

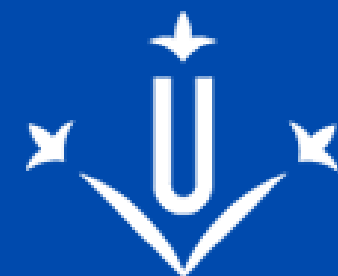
Applications for Mobile Devices

Remote Persistence

Dr. Vítor Luiz da Silva Verbel

vitor.dasilva@udl.cat

Course: 105025-2526



**Universitat
de Lleida**



Contents

Sprint 03 - Review

Introduction

Remote Persistence

Retrofit

Sprint 04

Before we go

Before we go

- Download at Virtual Campus: /code/[retrofitbooks.zip](#)
- Download at Virtual Campus: /code/[RemotePersistence.zip](#)
- Download Postman <https://www.postman.com/downloads/>

Sprint Review – Sprint 03

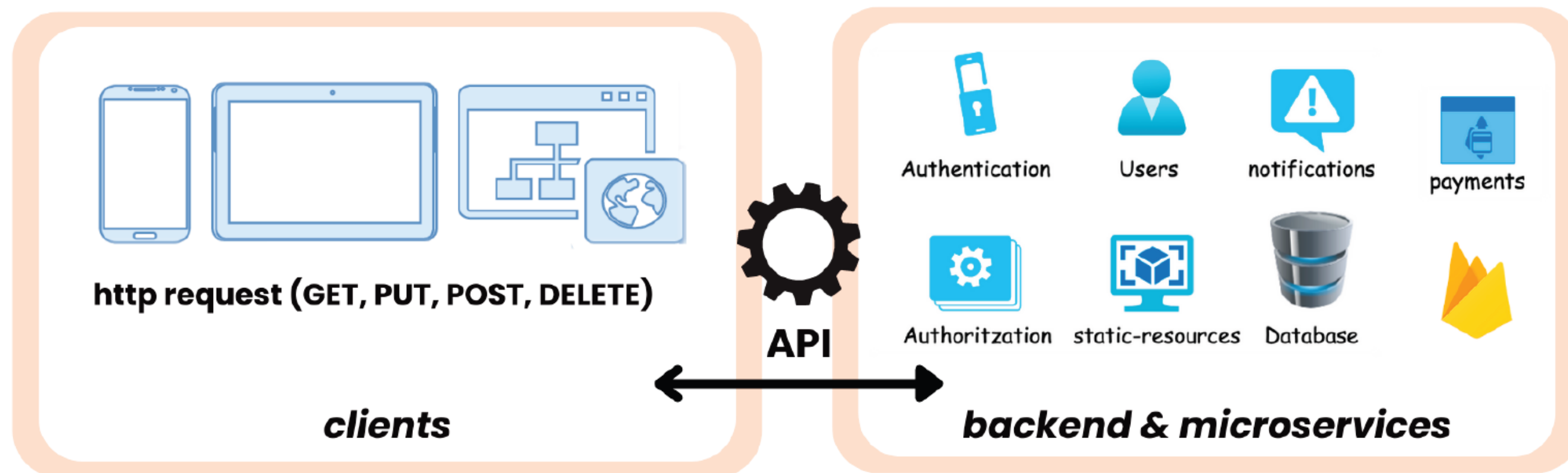
Introduction

Frontend and backend

In software engineering, the terms **frontend** and **backend** refer to the separation of concerns between the presentation layer (frontend) and the data access layer or the physical infrastructure or hardware.

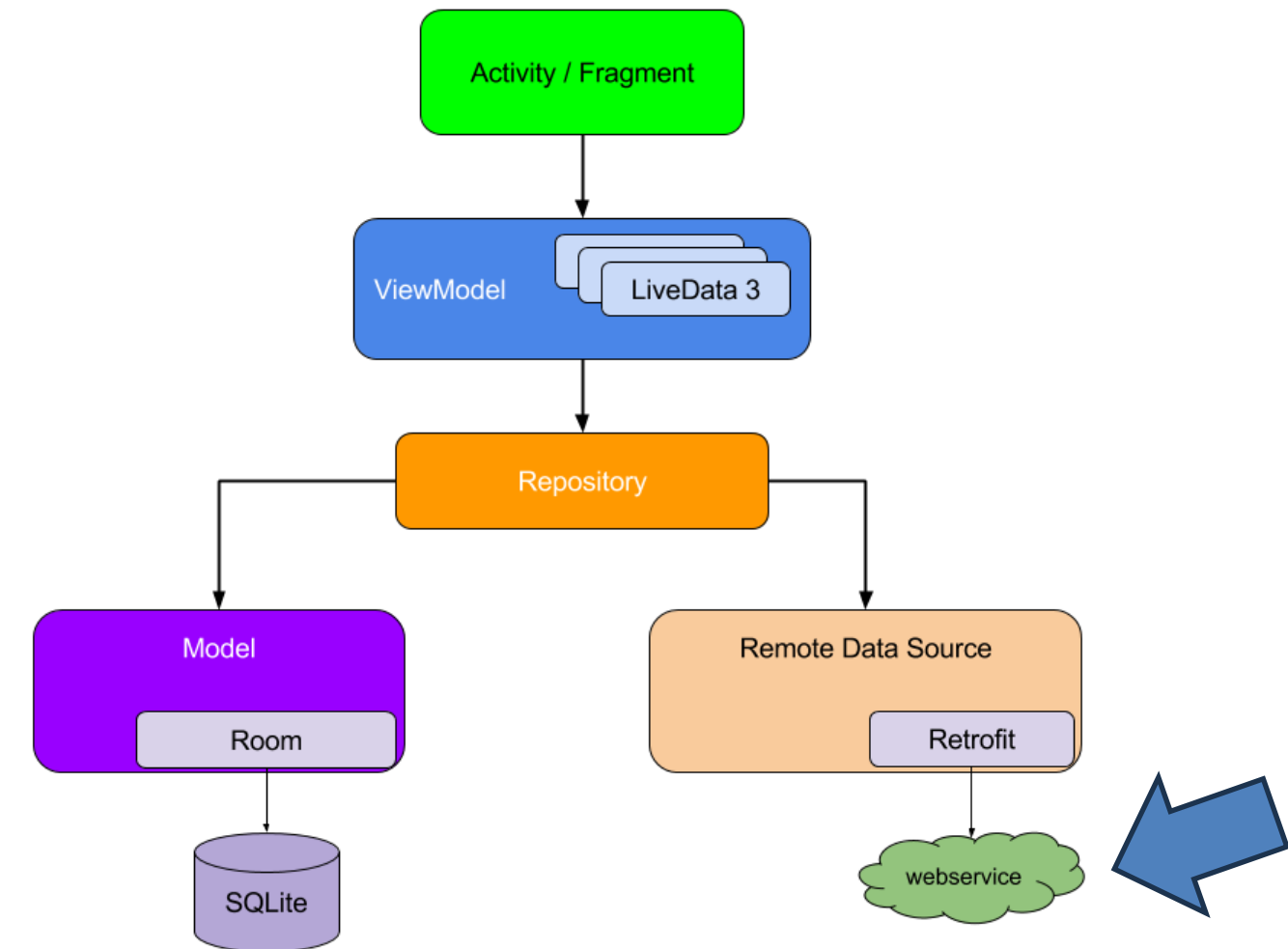
Exemple

In your project, the **frontend** is your Android application, while the **backend** will be a set of services that assist the backend, such as the operations with the database.



Application Programming Interfaces (API)

- **APIs** allow software running on different systems to communicate.
- **APIs** allow programs to read, write or delete variables or information from another system.
- It is common to use standard **API** interfaces for interoperability and avoid reinventing the wheel.



REpresentational State Transfer (REST) API Attributes

REST is an architectural style for providing standards between computer systems on the web, making it easier for systems to communicate with each other. These systems are characterized by being stateless and separating the concerns of client and server.

1. **Uniform Interface:** Decouple principal. You must define a contract between the client and the service using standards.
2. **Client-Server:** Enforce separation of concerns. Client and server evolve independently.
3. **Stateless:** each request from the client to the server must contain all of the information necessary to understand and complete the request. So, The server cannot take advantage of any previously stored context information.
4. **Cacheable:** The client application gets the right to reuse the response data later for equivalent requests and a specified period.
5. **Layered System:** The client is agnostic as to how many layers, if any, there are between the client and the actual server responding to the request (secure layer, cache layer, load balancer,...)
6. **Code-on-demand:** REST also allows client functionality to extend by downloading and executing code in the form of applets or scripts.

Communication between Client and Server

In the **REST** architectures, it is required that a client requests the server to retrieve or modify data on the server.

HTTP Request

HTTP stands for Hypertext Transfer Protocol and structures requests and responses over the internet. HTTP requires data to be transferred over the network from one point to another.

A request generally consists of:

- An HTTP verb, which defines what kind of operation to perform. (GET,PUT,POST,DELETE).
- A header allows the client to pass along information about the request.
- A path to a resource is an optional message body containing data.

REST Client Requests: CRUD

1. **CREATE:** Create a data structure or a variable and set this initial value.
2. **READ:** Query server for a variable.
3. **UPDATE:** Update content of a variable on the server.
4. **DELETE:** Delete the content of a variable on the server.

```
"book": {  
  "id": <Integer>,  
  "title": <String>,  
  "author": <String>,  
  "isbn": <Integer>  
}
```

CREATE

The client will supply (title, author and ISBN), and the server will create an instance of the book and add it to the library.

READ

The client wants to see a list of all the books in the library. Also, we would have a function to filter book by author or retrieve a single book.

UPDATE

The client wants to supply new values (title, author and ISBN).

REMOVE

The client will supply the value to identify a book and delete it from the library.

Combining REST and HTTP requests

- GET — retrieve a specific resource (by id) or a collection of resources
- POST — create a new resource
- PUT — update a specific resource (by id)
- DELETE — remove a specific resource by id

CRUD	REST
CREATE	POST
READ	GET
UPDATE	PUT,PATCH
DELETE	DELETE

Combining REST and HTTP responses

Status	Meaning
200 (OK)	This is the standard response for successful HTTP requests.
201 (CREATED)	This is the standard response for an HTTP request that resulted in an item being successfully created.
204 (NO CONTENT)	This is the standard response for successful HTTP requests, where nothing is being returned in the response body.
400 (BAD REQUEST)	The request cannot be processed because of wrong request syntax, excessive size, or another client error.
403 (FORBIDDEN)	The client does not have permission to access this resource.
404 (NOT FOUND)	The resource could not be found at this time. It is possible it was deleted or does not exist yet.
500 (INTERNAL SERVER ERROR)	The generic answer for an unexpected failure if there is no more specific information available.

Example: Creating a book

```
POST http://damcore/book/  
Body:  
{  
  "book": {  
    "title": "One Piece Volume 1",  
    "author": "Eiichiro Oda",  
    "isbn": 0782009136637  
  }  
}
```

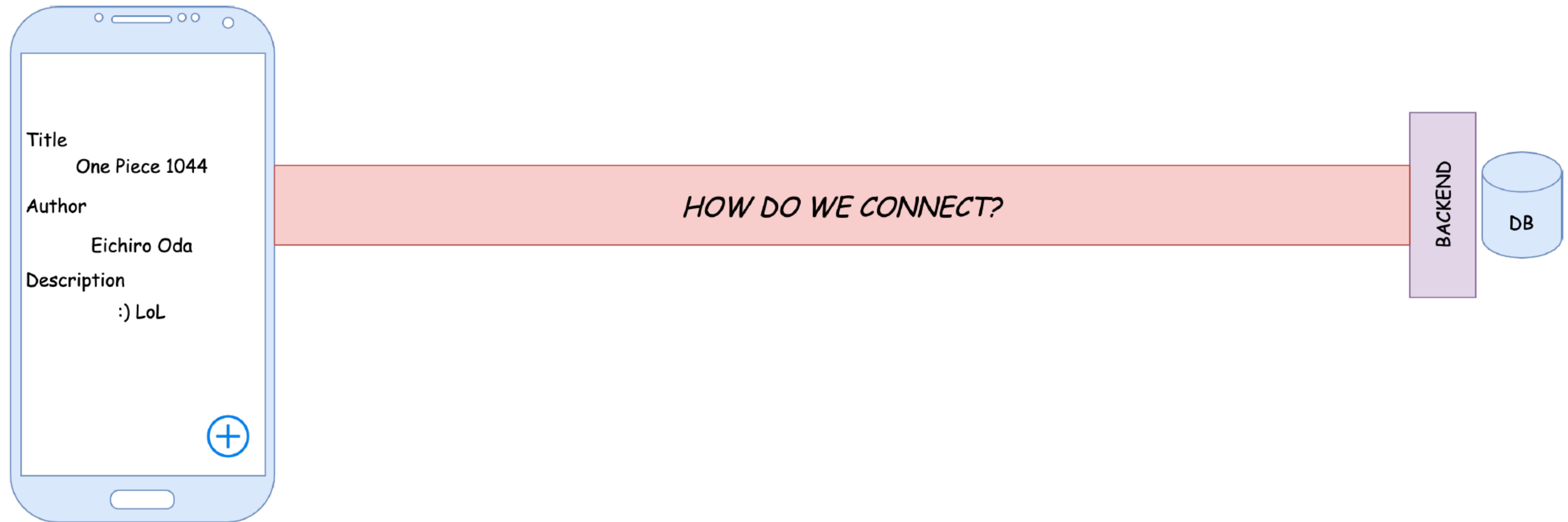
```
Status Code - 201 (CREATED)  
Body -  
{  
  "book": {  
    "id":1,  
    "title": "One Piece Volume 1",  
    "author": "Eiichiro Oda",  
    "isbn": 0782009136637  
  }  
}
```

Design TIPS

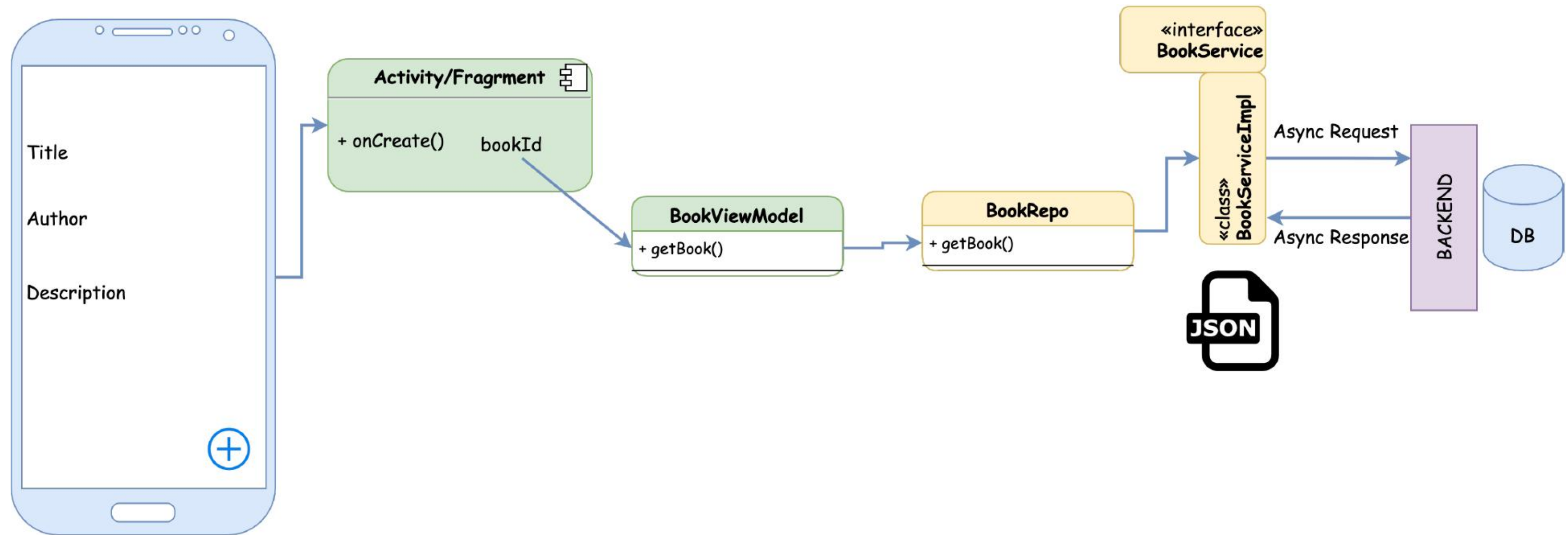
- What routes would you need to implement to provide your workout class model with this CRUD functionality, and what are their corresponding HTTP verbs?
- What effect would each route have on the database?
- What response body would each route return?
- What response code would each route return?

Remote Persistence

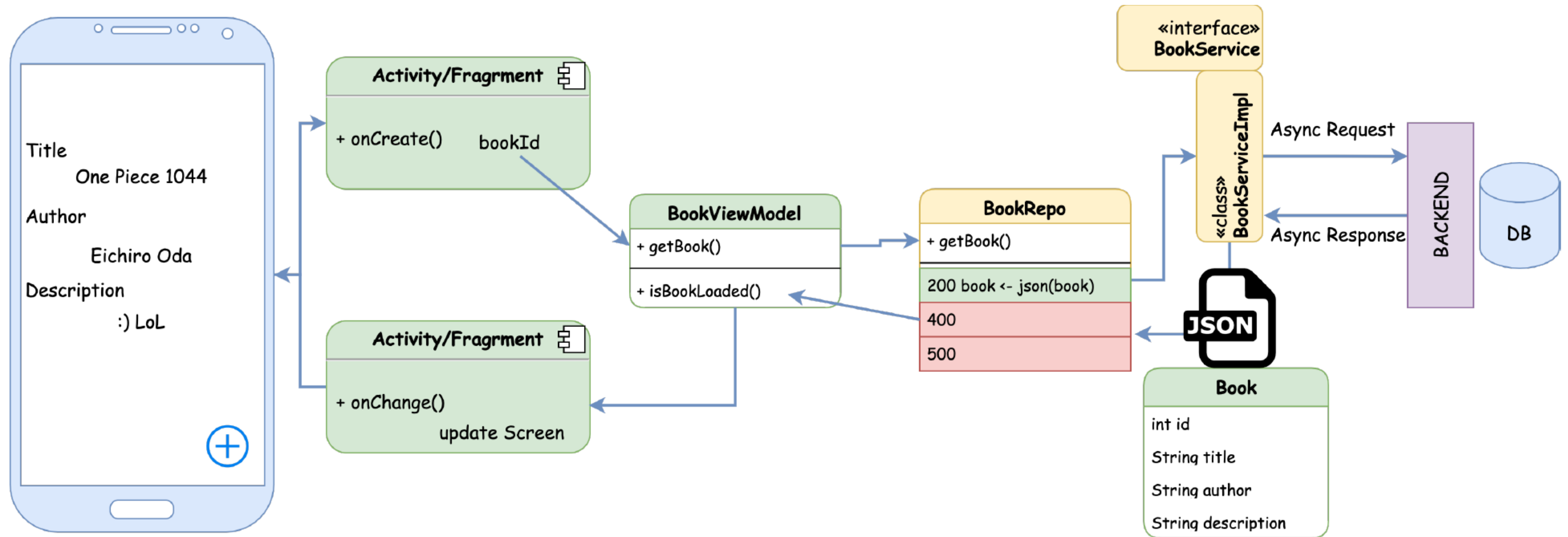
What is the problem?



Client - Backend (Communication)



Backend - Client (Communication)



To solve the problem... we need to learn...

Asynchronous/synchronous

Asynchronous requests should be preferred to **synchronous** requests for performance reasons. **Synchronous** requests block the execution of code, which causes *freezing* on the screen and an **unresponsive user experience**.

Mapping

Mapping JSON response to class instances in java

Flow

How can we organize to let the information flow straightforwardly?

Retrofit

What is Retrofit?

Overview

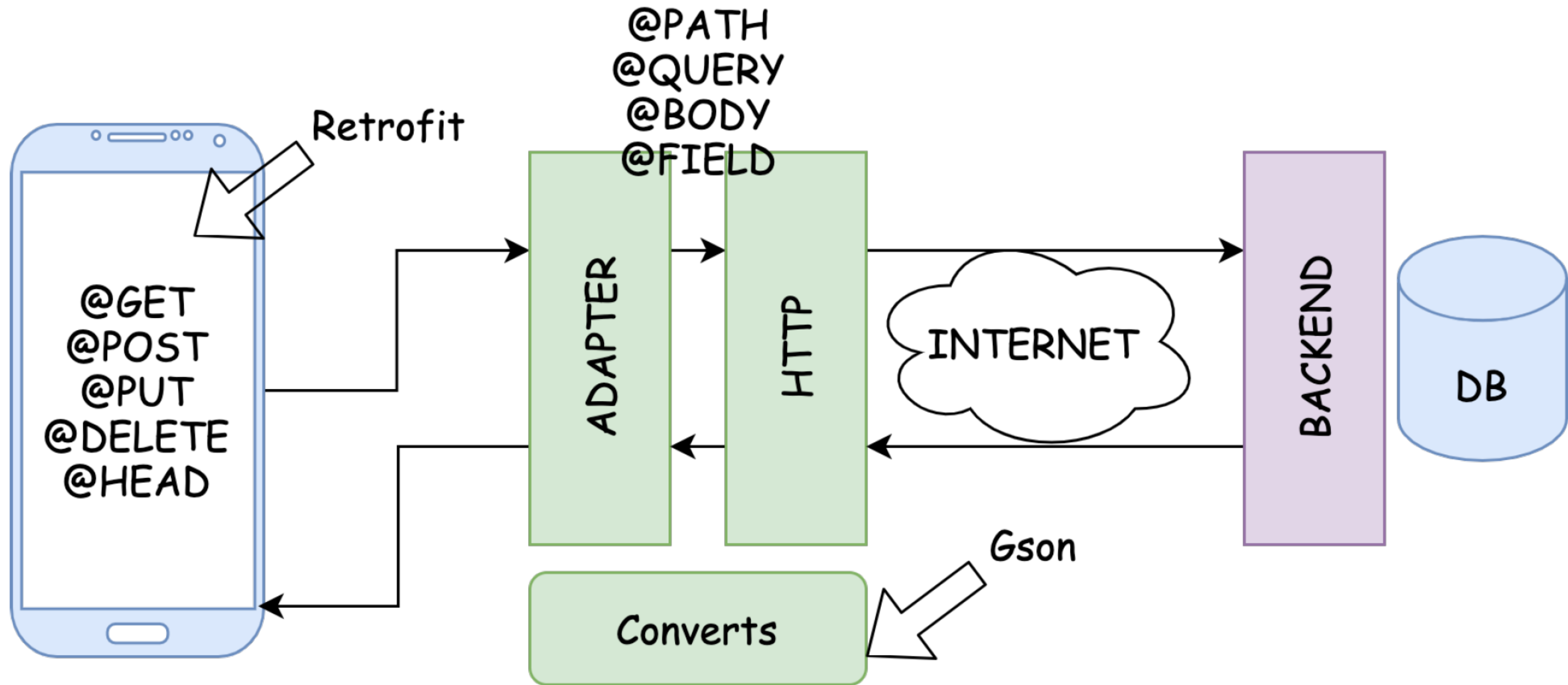
Retrofit is a *REST Client for Java and Android* by Square inc under Apache 2.0 license. It's an *easy and fast library* to **retrieve and upload the data** (JSON or other structured data) via a REST-based web service. It's a simple network library that is used for network transactions. We can seamlessly capture JSON responses from web service/web API using this library.

⇒ This library makes downloading **JSON or XML** data from a web API reasonably straightforward. Once the data is downloaded, it is parsed into a *Plain Old Java Object (POJO)*, which must be defined for each “resource” in the response.

Book

- Book json ⇒ is transformed into ⇒ Book class ⇒ `Book book = new Book()`
- $\forall key, value \in json \Rightarrow$ `book.setKey(value)`

How does Retrofit fit your goal?



How does it work?

Retrofit turns your HTTP API into a Java interface.

```
public interface GitHubService {  
    @GET("users/{user}/repos")  
    Call<List<Repo>> listRepos(@Path("user")  
        String user);  
}
```

In this example, we are interfacing a **GET** operation asking for all the repositories for a specific **user**. Note the **{user}** is a variable, and it can be set in *runtime*. Moreover, this interface indicates that the JSON object received from the request will be a map to a JAVA data struct (a list) of the class Repo. We can map a **JSON** to an **Object** automatically and transparently.

Essential Annotations

Annotation	Description
@Path	variable substitution for the API endpoint (i.e. username will be swapped for {username} in the URL endpoint).
@Query	specifies the query key name with the value of the annotated parameter.
@Body	payload for the POST call (serialized from a Java object to a JSON string)
@Header	specifies the header with the value of the annotated parameter

Headers

```
@Headers({"Cache-Control: max-age=640000", "User-Agent:{}"})  
@GET("xxxxxxxxx")
```

POST and Forms

Adding Forms

```
@FormUrlEncoded
@POST("xxxxxxx")
Observable<SomeResponse> getSth(@Field("property") String property);
```

MultiForms

```
@Multipart
@POST("xxxxxxx")
Call<Response> uploadImage(@Part("description")
    String description, @Part("image") RequestBody image)
```

Book Sample

```
public class Book {  
    @SerializedName("id")  
    private Integer id;  
    @SerializedName("tittle")  
    private String tittle;  
    @SerializedName("description")  
    private String description;  
    @SerializedName("author")  
    private String author;  
}
```

```
{  
    "tittle": "One Piece 1044",  
    "author": "Eichiro Odo",  
    "description": ":) Lol"  
}
```

```
public interface BookService {  
    @GET("/book/show")  
    Call<Book> getBookById(@Query("id")  
        int id);  
}
```

The retrofit will send the petition to the backend via the internet. We ask the backend to obtain information of a specific book (*/book/show*), and we use a query parameter **@Query("id")**. Our backend will search in the DB and generate the **JSON**, once retrofit receives the JSON, then uses gson to map each property to an instance of the class Book! ⇒ Note: The **@SerializedName("id")** needs to be the same as the JSON key.

Book Sample (Kotlin)

DTO

```
data class BookDto(  
    val id: Int?,  
    val title: String,  
    val description: String,  
    val author: String,  
)
```

Domain

```
data class Book(  
    val id: Int?,  
    val title: String,  
    val description: String,  
    val author: String,  
)
```

Book sample (Book Service)

```
public interface BookService {  
    @GET("/book/show")  
    Call<Book> getBookById(@Query("id") int id);  
    @POST("/book/create")  
    Call<ResponseBody> createBook(@Body Book book);  
    @PUT("/book/update")  
    Call<ResponseBody> updateBookById(@Query("id") int id,  
                                       @Body Book book);  
    @DELETE("/book/delete")  
    Call<ResponseBody> deleteBookById(@Query("id") int id);  
    @GET("/books")  
    Call<List<Book>> getBooks(@Header("Authorization") String token);  
}
```


Book sample (Book Service) (Kotlin)

```
interface BookApiService {

    @GET("{group_id}/book")
    suspend fun getBookById(@Path("group_id") groupId: String, @Query("id") id: Int): BookDto

    @POST("{group_id}/book/create")
    suspend fun createBook(@Path("group_id") groupId: String, @Body book: BookDto): ResponseBodyDto

    @PUT("{group_id}/book/update")
    suspend fun updateBook(
        @Path("group_id") groupId: String,
        @Query("id") id: Int,
        @Body book: BookDto
    ): ResponseBodyDto

    @DELETE("{group_id}/book/delete")
    suspend fun deleteBook(@Path("group_id") groupId: String, @Query("id") id: Int): ResponseBodyDto

    @GET("{group_id}/books")
    suspend fun getBooks(@Path("group_id") groupId: String): List<BookDto>
}
```

Dependencies

Gradle.kts

```
implementation("com.squareup.retrofit2:retrofit:2.9.0")  
implementation("com.squareup.retrofit2:converter-gson:2.9.0")
```

AndroidManifest.xml

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Build

```
Retrofit.Builder()  
    .baseUrl("http://13.39.162.212:8080/")  
    .addConverterFactory(GsonConverterFactory.create())  
    .client(  
        OkHttpClient.Builder()  
            .addInterceptor(HttpLoggingInterceptor().apply {  
                level = HttpLoggingInterceptor.Level.BODY  
            })  
            .build()  
    )  
    .build()
```

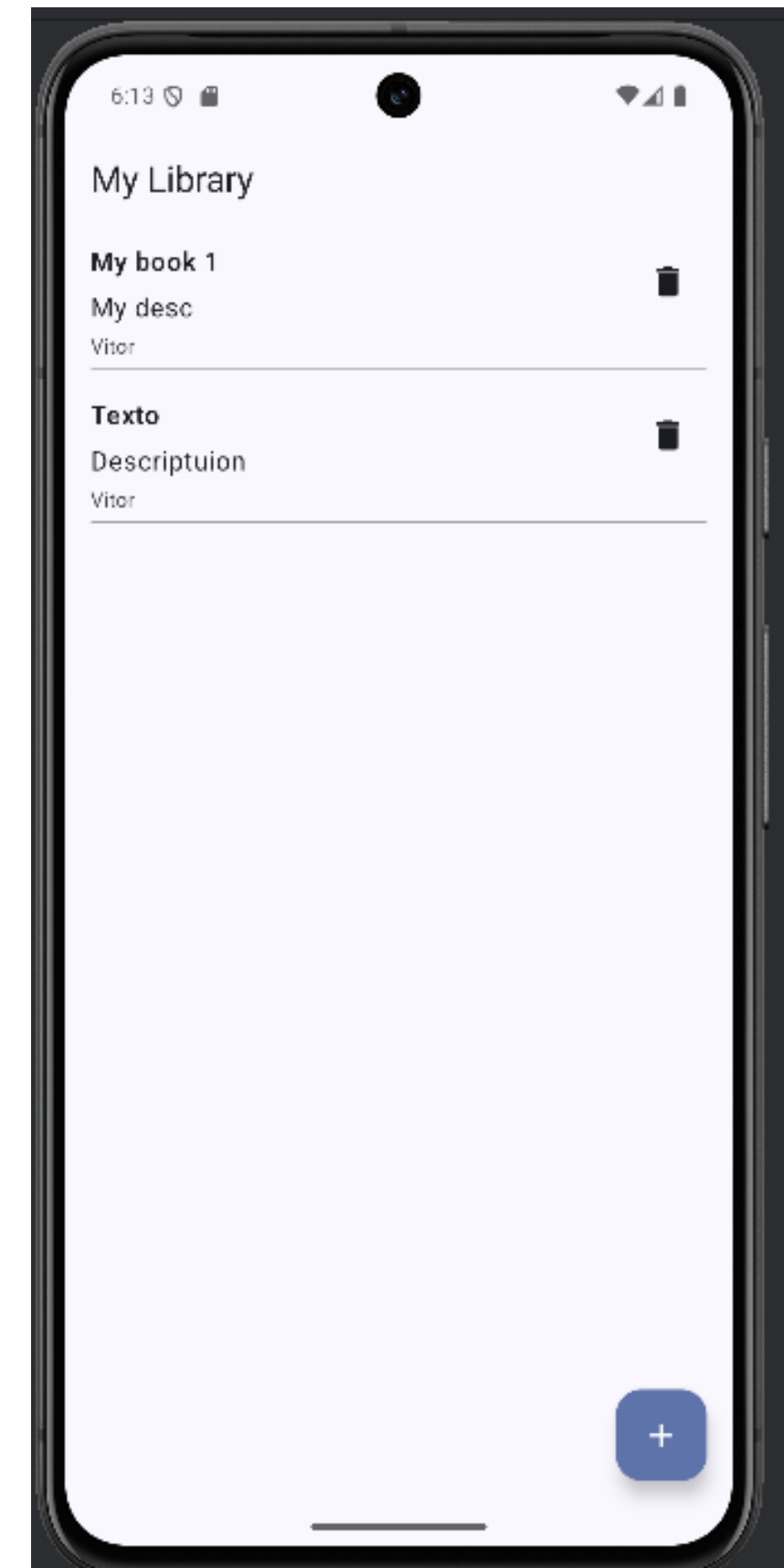
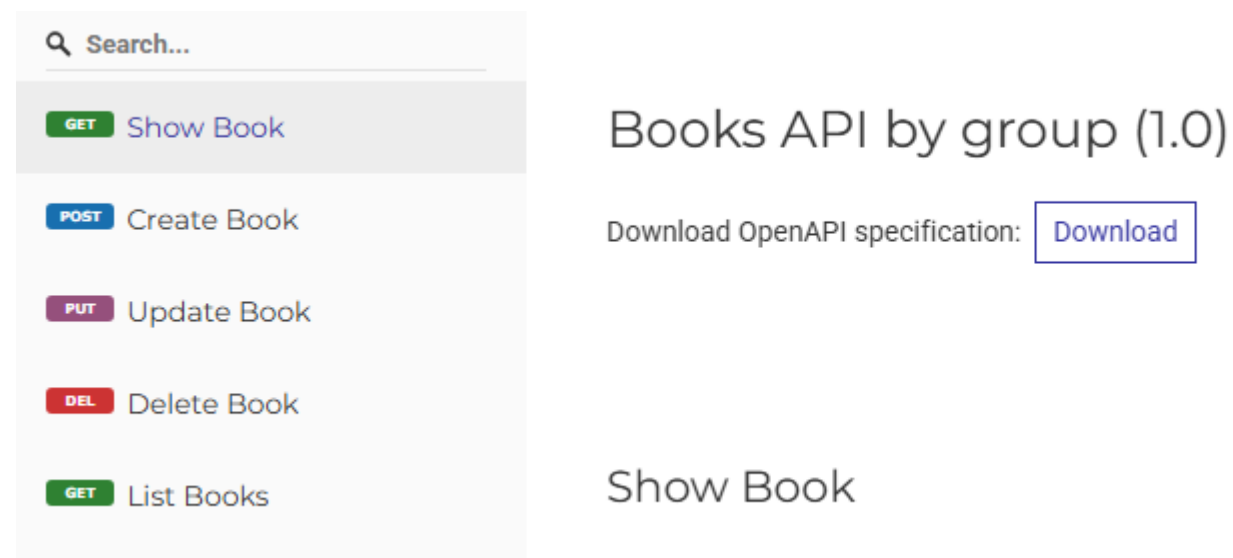

Hands On – Books APP

Code Review

- Download /code/[retrofitbooks.zip](#)
- Download Postman <https://www.postman.com/downloads/>

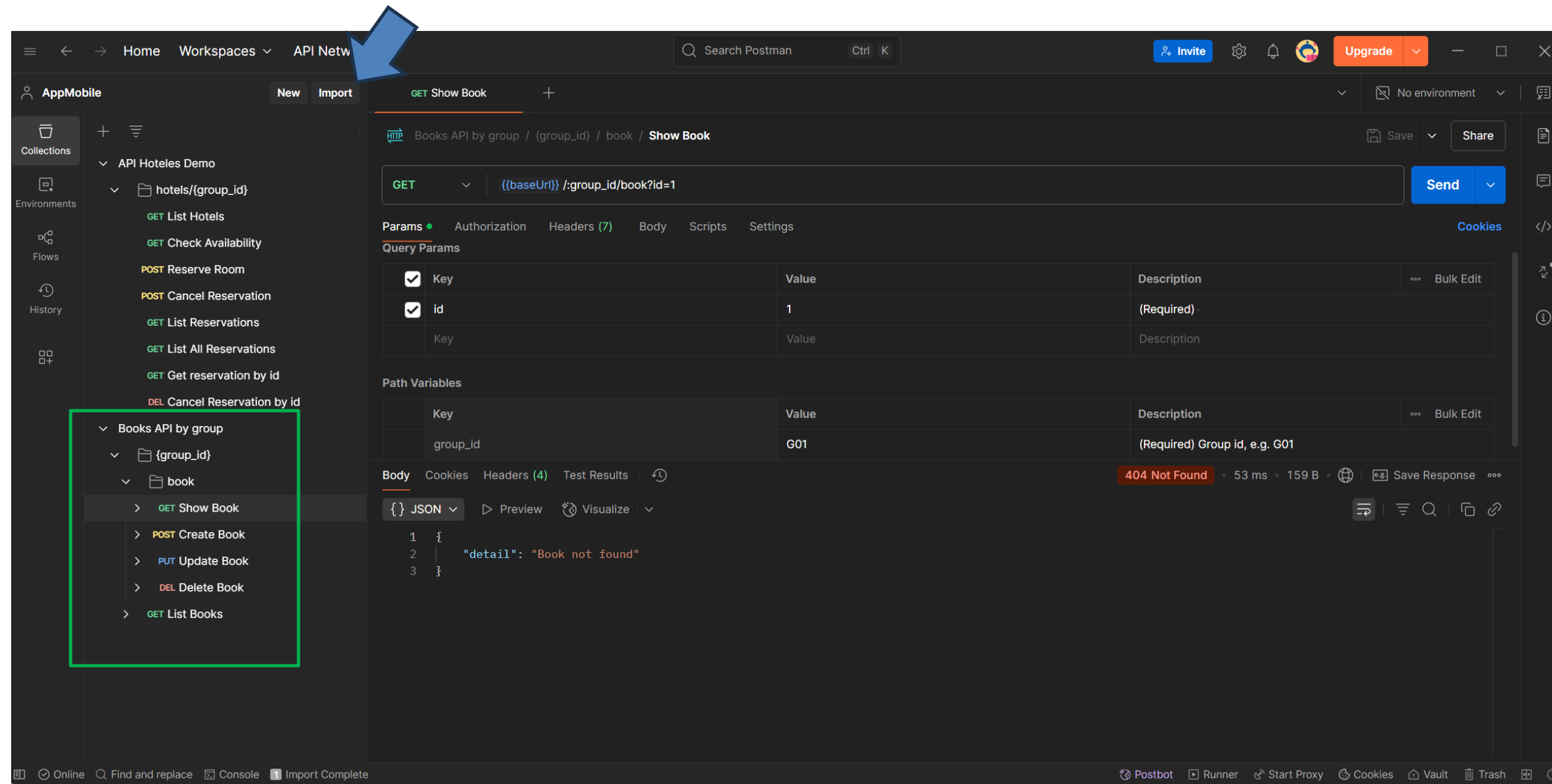
Book API:

- URL: <http://15.224.84.148:8080/>
- OpenApi: <http://15.224.84.148:8080/openapi.json>
- Documentation:
 - [http:// 15.224.84.148:8080 /docs](http://15.224.84.148:8080/docs)
 - [http:// 15.224.84.148:8080 /redoc](http://15.224.84.148:8080/redoc)



Open Postman

- Import openapi.json file: <http://15.224.84.148:8080/openapi.json>
- You should see a list of methods in Books API



Books API

- If you open <http://15.224.84.148:8080/docs> you will see a list of method available.

Books API by group ^{1.0} ^{OAS 3.1}

/openapi.json

default ^

GET `/group_id/book` Show Book

POST `/group_id/book/create` Create Book

PUT `/group_id/book/update` Update Book

DELETE `/group_id/book/delete` Delete Book

GET `/group_id/books` List Books

Books API

- You can consume this API in your browser, just clicking in Try it Out.

default ^

GET `/[{group_id}]/book` Show Book ✓

POST `/[{group_id}]/book/create` Create Book  

Parameters

Name	Description
group_id * required string (path)	group_id

Necessary parameter to identify which group it belongs

Request body **required** application/json

Example Value | Schema

```
{
  "id": 0,
  "title": "string",
  "description": "string",
  "author": "string"
}
```


GROUPS IDs

- From now on, use this code in your API requests:

Grupo	Estudiante 1	Estudiante 2
G01	Acosta Contreras, Sharon Elizabeth	Sánchez Macià, Joan
G02	Banqué Torné, Nel	Guarro Pérez, Gerard
G03	Beruet Morelli, Marco	Gil Cañizares, Ivan
G04	Cribillers Moncunill, Arnau	Lázaro Roca, Paul
G05	El Kabiri Ouarqui, Anouar	Mora Palomino, Eloi
G06	Escofet Presedo, Àlex	Gomà Atehortua, Samuel
G07	Garcia Huanca, Edson David	
G08	Giralt Padró, Xavier	Obando Coca, Benjamin Jonas
G09	Jaén Rico, Izan	
G10	Juárez Ontiveros, Asier	
G11	Minguez Arroyo, Adria	
G12	Alcoverro López, Guillem	Molinero Quintana, Nil
G13	Ràfols Bayot, Aleix	
G14	Senar Tejedor, Alba	
G15	Talayero Carrasco, Guillem	Àlvarez I Arisa, Oriol
G16	Rubio Palomo, Raúl	Vazquez Sanchez, Iker
G17	Farriols Segura, Guillem	

Testing using Online Documentation (FastAPI)

- CREATE

POST `/[group_id]/book/create` Create Book

Parameters Cancel Reset

Name	Description
group_id * required string (path)	G99

Request body required application/json

```
{
  "title": "My book 1",
  "description": "Desc book 1",
  "author": "Vitor da Silva"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://13.39.162.212:8080/G99/book/create' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "My book 1",
    "description": "Desc book 1",
    "author": "Vitor da Silva"
  }'
```

Request URL

`http://13.39.162.212:8080/G99/book/create`

Server response

Code	Details
201	Response body <pre>{ "ok": true, "message": "Book created", "book": { "id": 3, "title": "My book 1", "description": "Desc book 1", "author": "Vitor da Silva" } }</pre> Response headers <pre>content-length: 127 content-type: application/json date: Tue, 29 Apr 2025 06:39:38 GMT server: uvicorn</pre>

Testing using Online Documentation (FastAPI)

- LIST

GET /{group_id}/books List Books

Parameters

Name	Description
group_id * required string (path)	G99

Execute

Clear

Cancel

Responses

Curl

```
curl -X 'GET' \
'http://13.39.162.212:8080/G99/books' \
-H 'accept: application/json'
```

Request URL

```
http://13.39.162.212:8080/G99/books
```

Server response

Code	Details
200	Response body <pre>[{ "id": 3, "title": "My book 1", "description": "Desc book 1", "author": "Vitor da Silva" }]</pre> Download

Testing using Online Documentation (FastAPI)

- GET BY ID

GET `/[{group_id}]/book` Show Book

Parameters

Name	Description
group_id * required string (path)	Group id, e.g. G01 G99
id * required integer (query)	3

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'http://13.39.162.212:8080/G99/book?id=3' \
-H 'accept: application/json'
```

Request URL

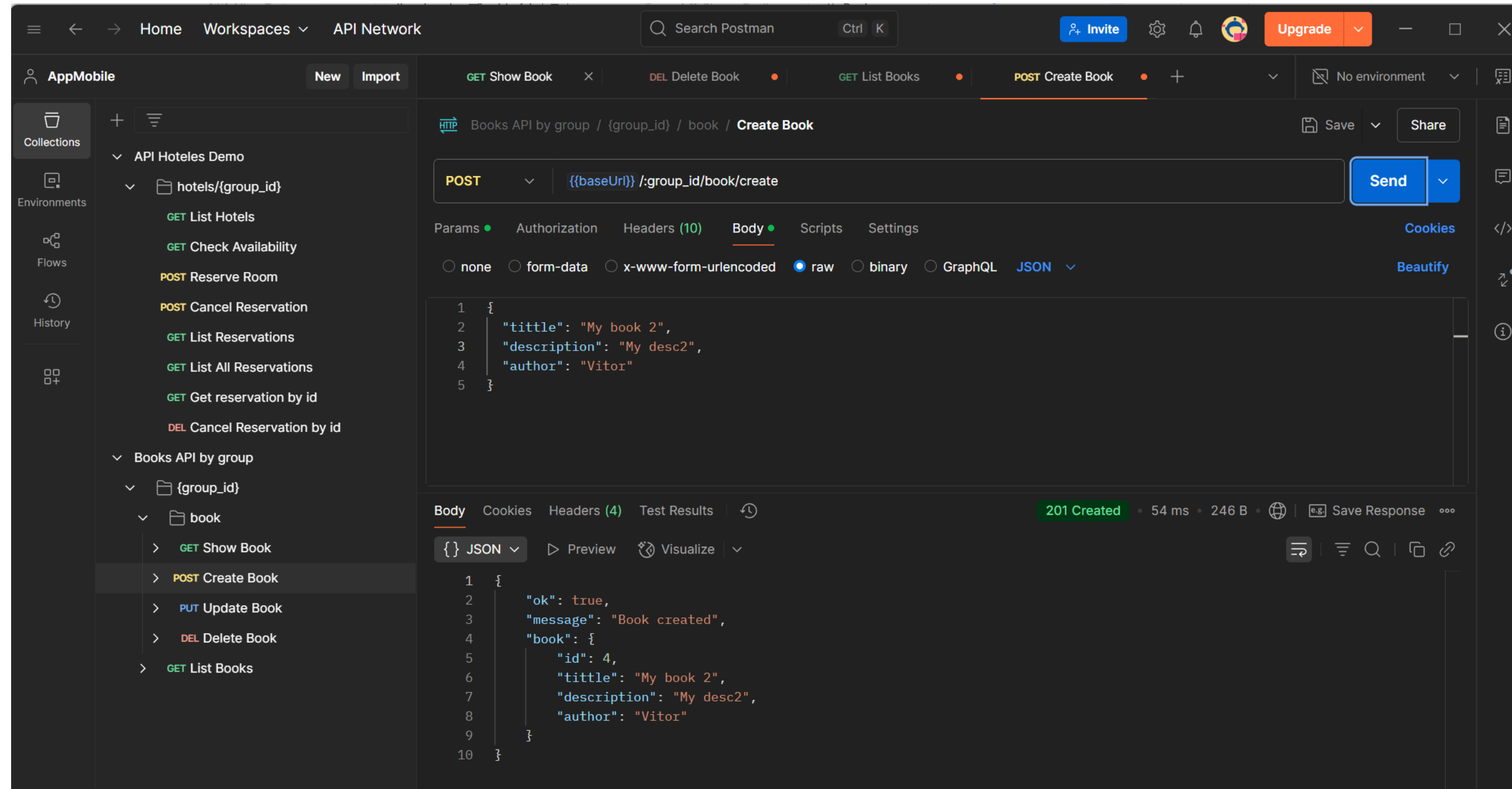
```
http://13.39.162.212:8080/G99/book?id=3
```

Server response

Code	Details
200	Response body <pre>{ "id": 3, "title": "My book 1", "description": "Desc book 1", "author": "Vitor da Silva" }</pre> Response headers <pre>content-length: 83 content-type: application/json date: Tue, 29 Apr 2025 06:43:35 GMT server: uvicorn</pre>

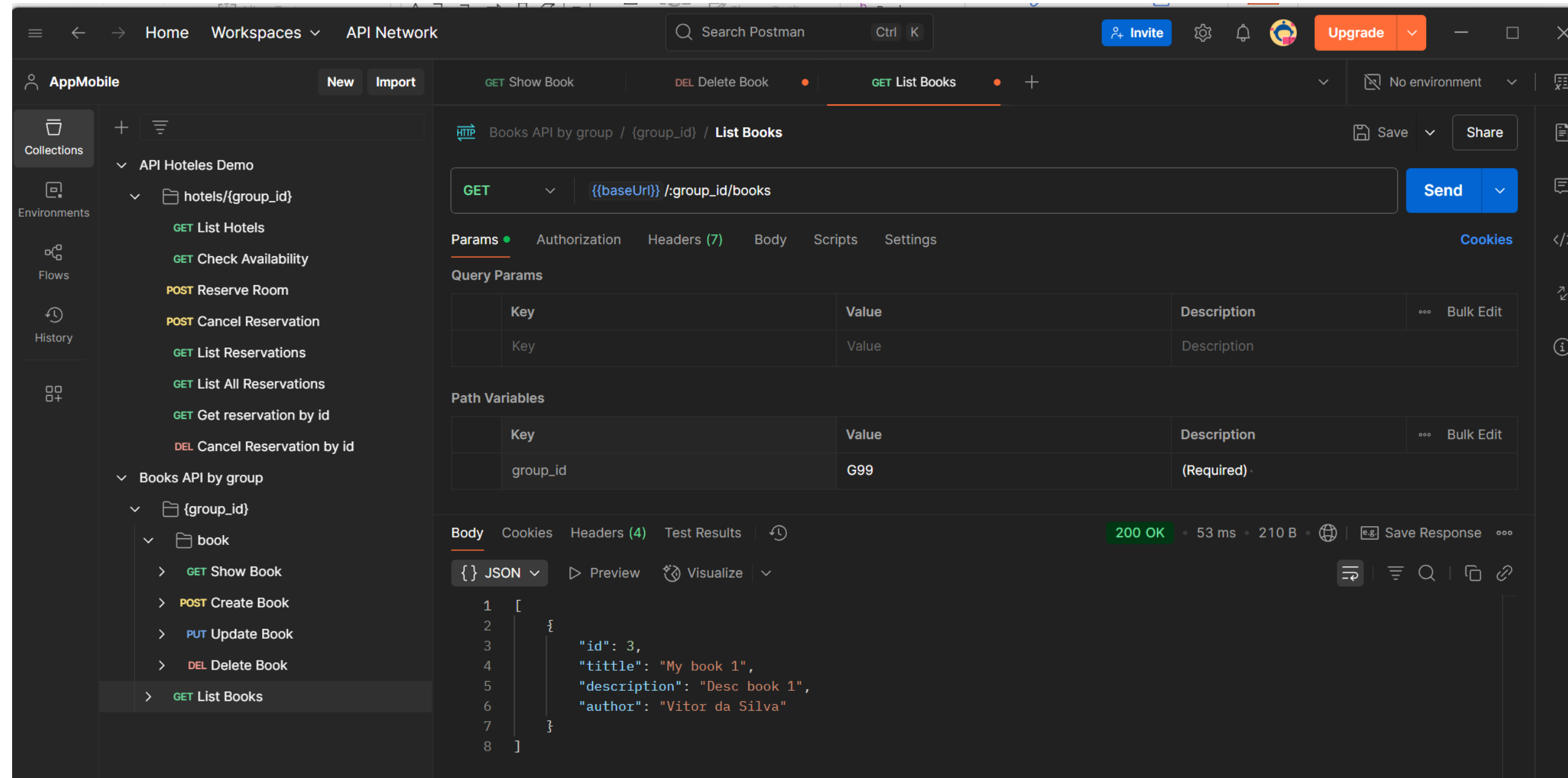
Testing using Postman

- CREATE



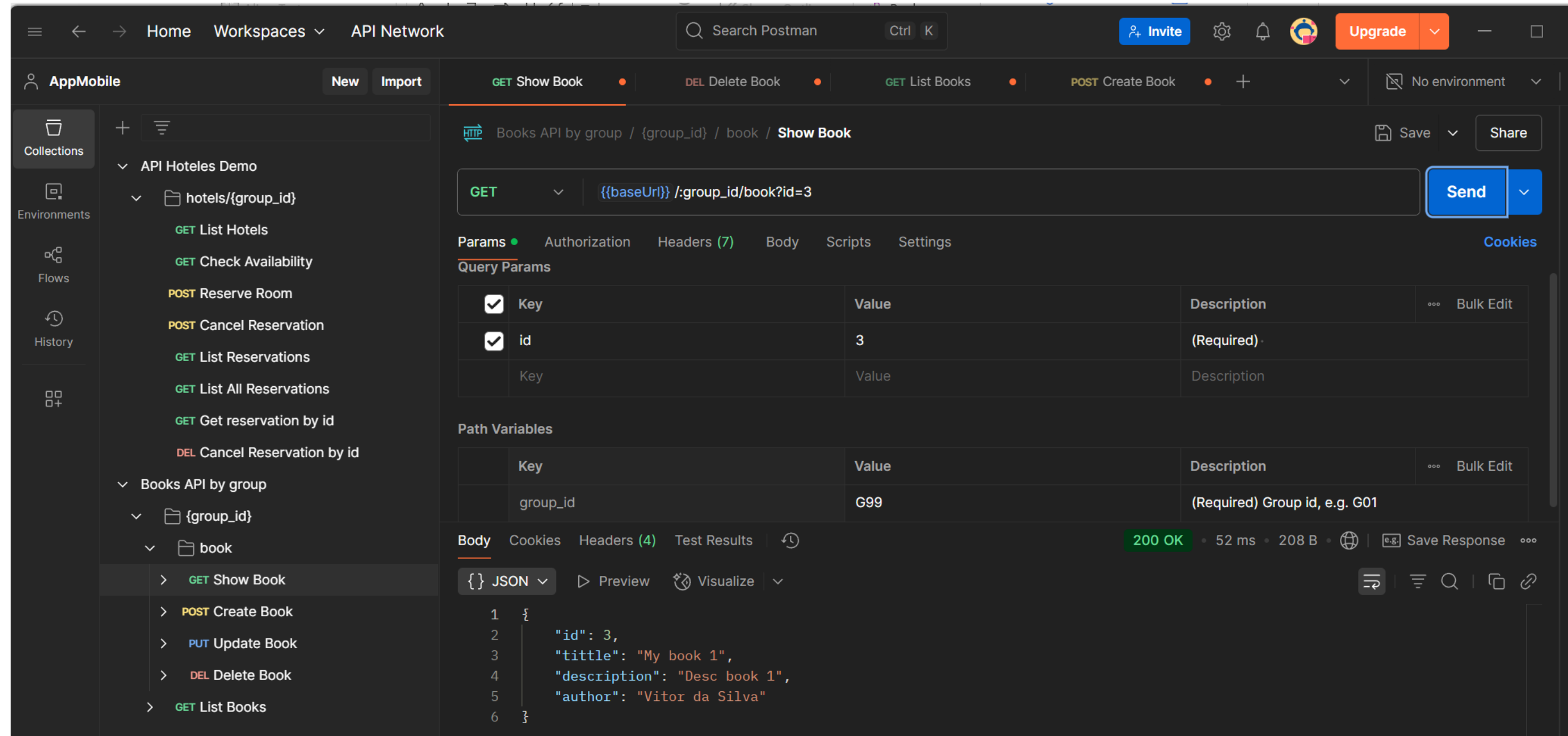
Testing using Postman

- LIST

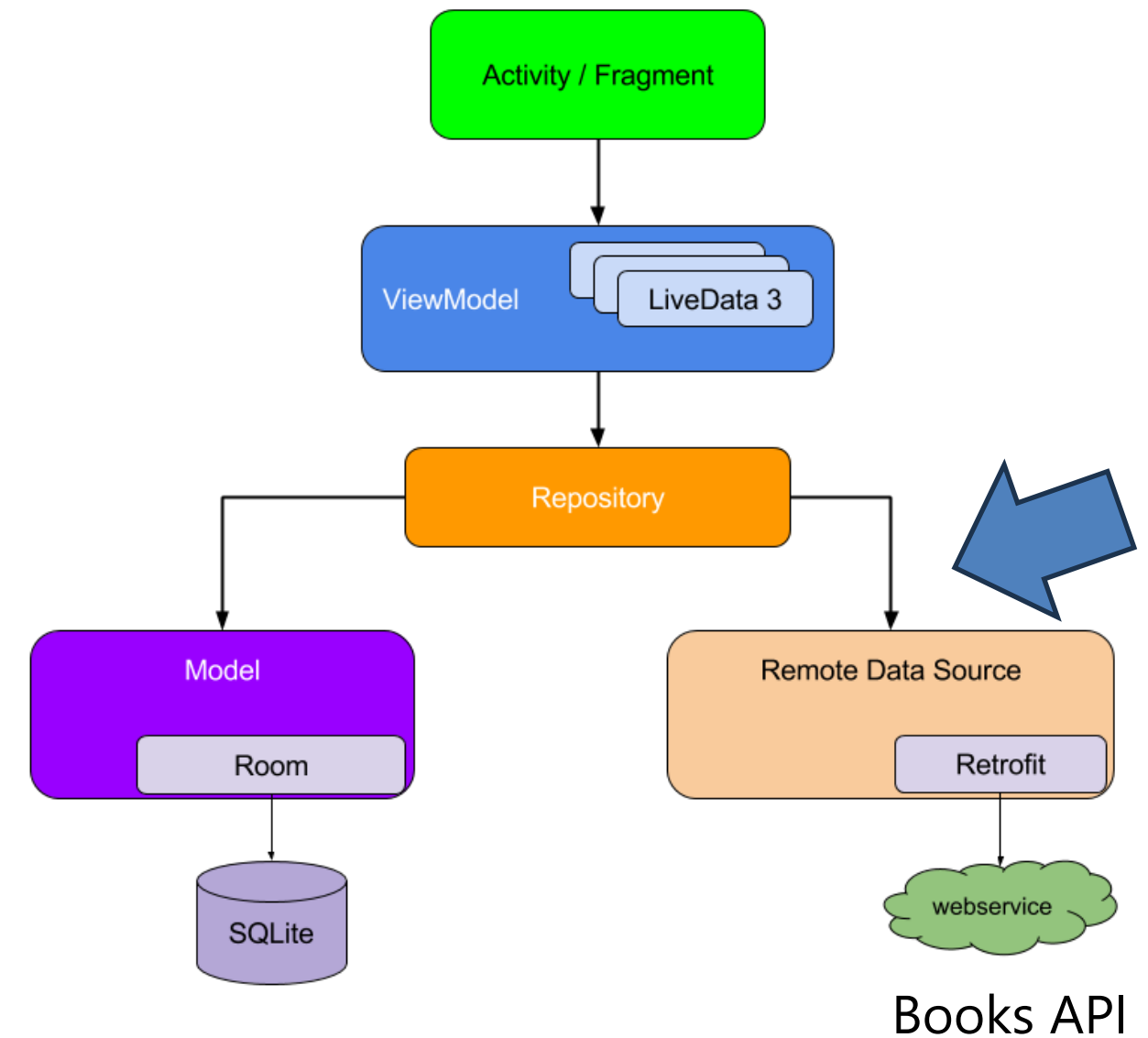
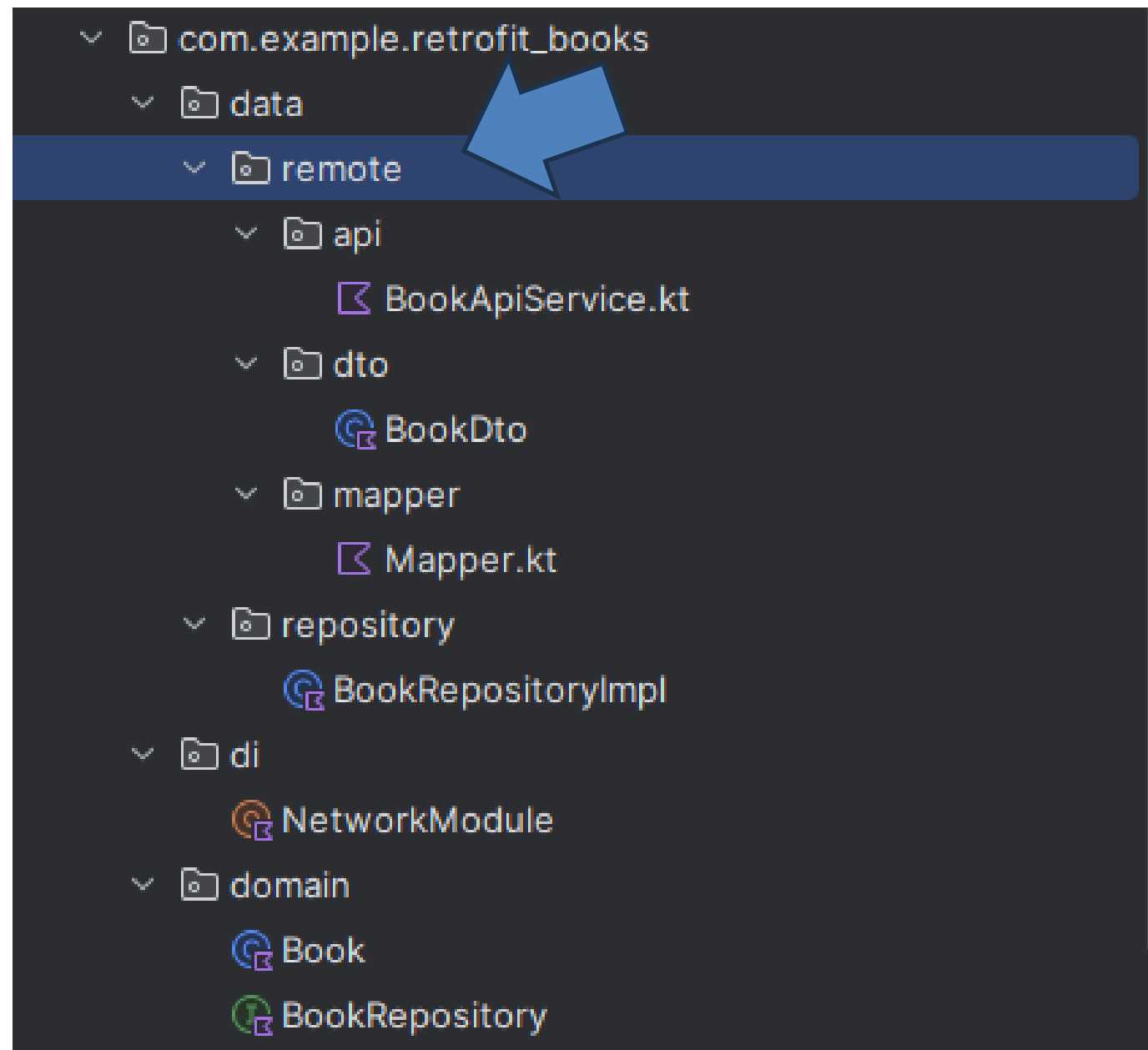


Testing using Postman

- GET BY ID



Books App - Code Review (Structure)



Books App - Code Review (Dependencies)

```
/* --- network --- */  
implementation("com.squareup.retrofit2:retrofit:2.9.0")  
implementation("com.squareup.retrofit2:converter-gson:2.9.0")  
implementation("com.squareup.okhttp3:logging-interceptor:5.0.0-alpha.11")
```


Books App - Code Review (Api Service)

```
interface BookApiService {

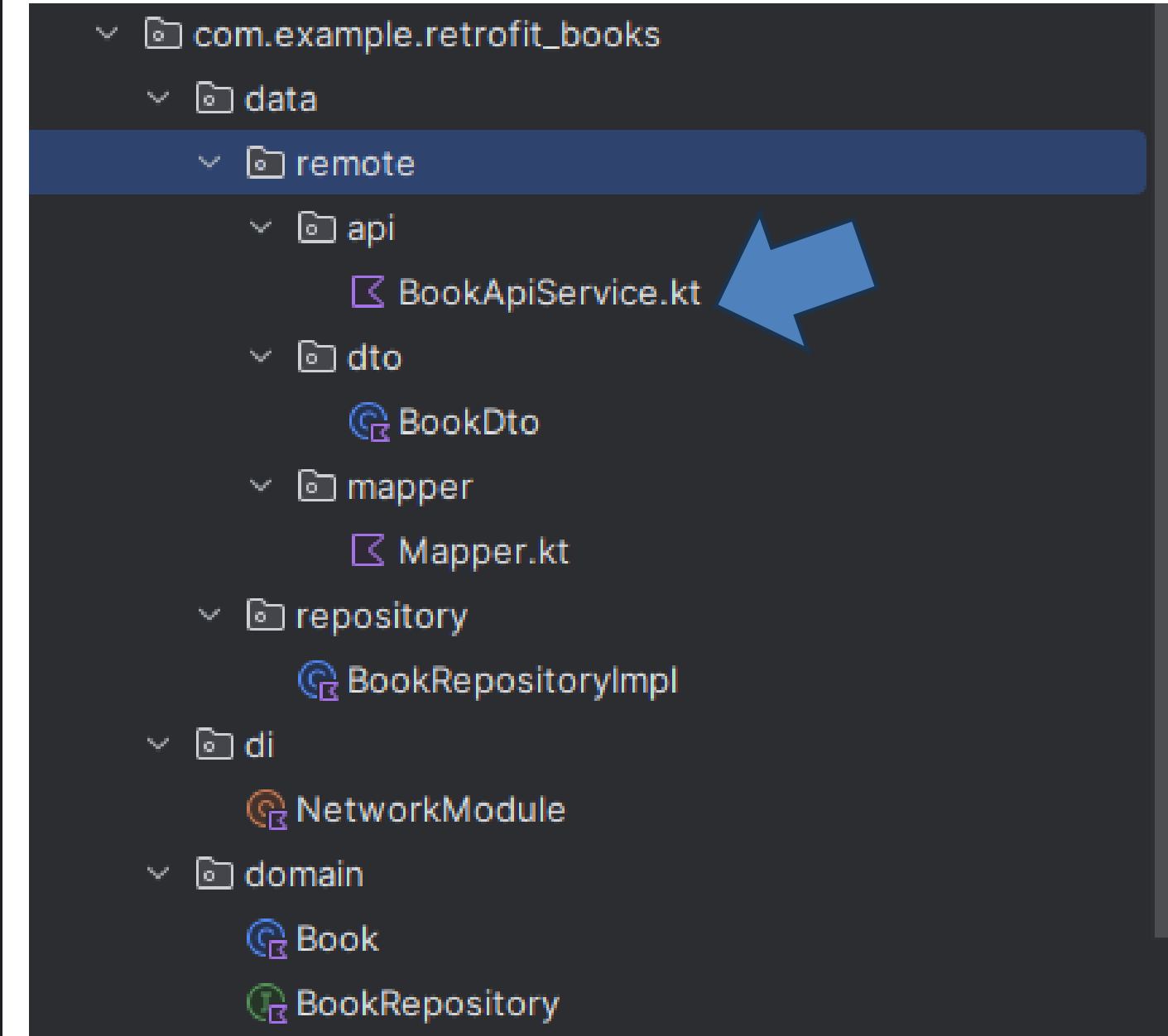
    @GET("{group_id}/book")
    suspend fun getBookById(@Path("group_id") groupId: String, @Query("id") id: Int): BookDto

    @POST("{group_id}/book/create")
    suspend fun createBook( @Path("group_id") groupId: String, @Body book: BookDto):
    ResponseBodyDto

    @PUT("{group_id}/book/update")
    suspend fun updateBook(
        @Path("group_id") groupId: String,
        @Query("id") id: Int,
        @Body book: BookDto
    ): ResponseBodyDto

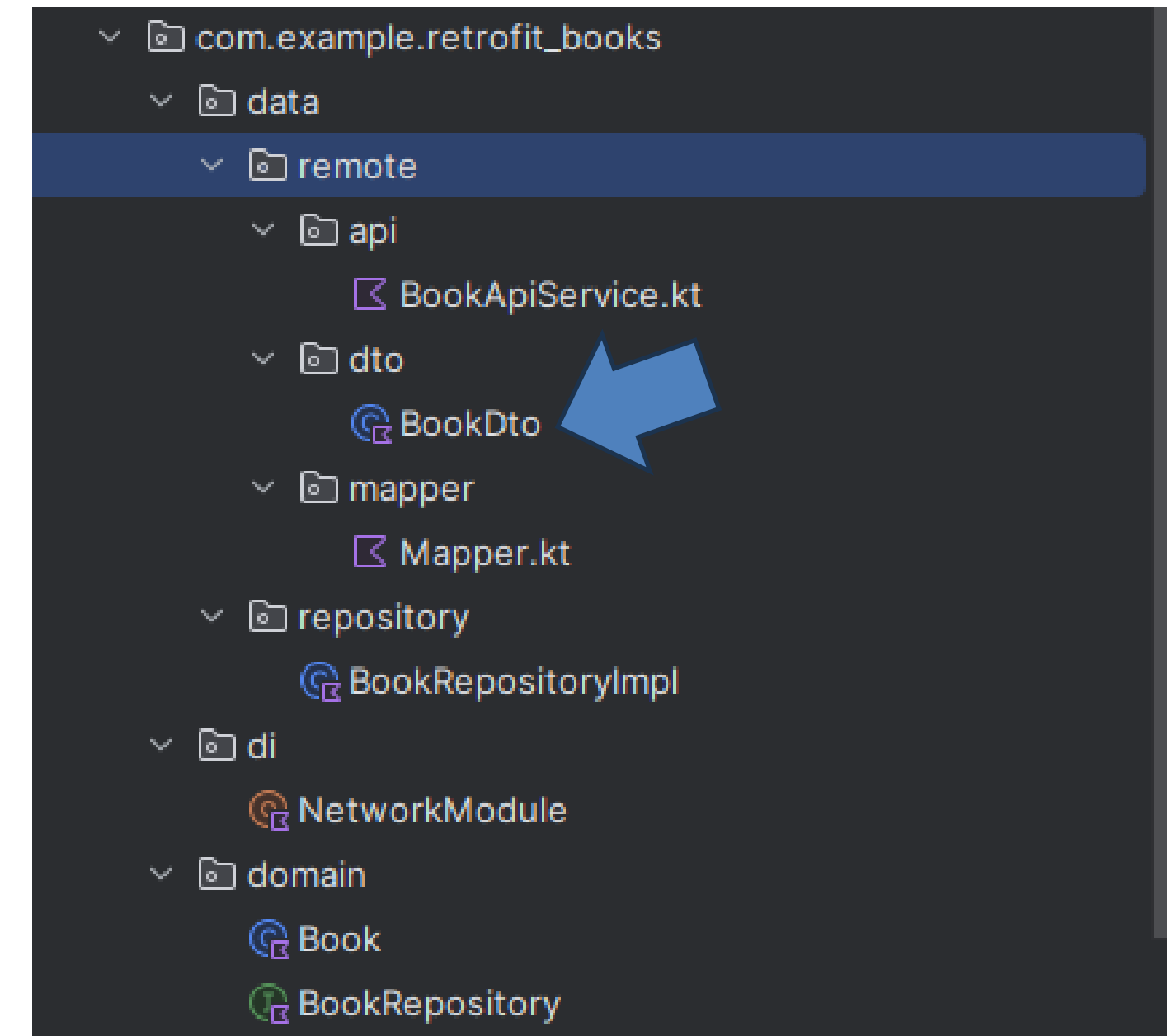
    @DELETE("{group_id}/book/delete")
    suspend fun deleteBook(@Path("group_id") groupId: String, @Query("id") id: Int):
    ResponseBodyDto

    @GET("{group_id}/books")
    suspend fun getBooks(@Path("group_id") groupId: String): List<BookDto>
}
```



Books App - Code Review (DTO)

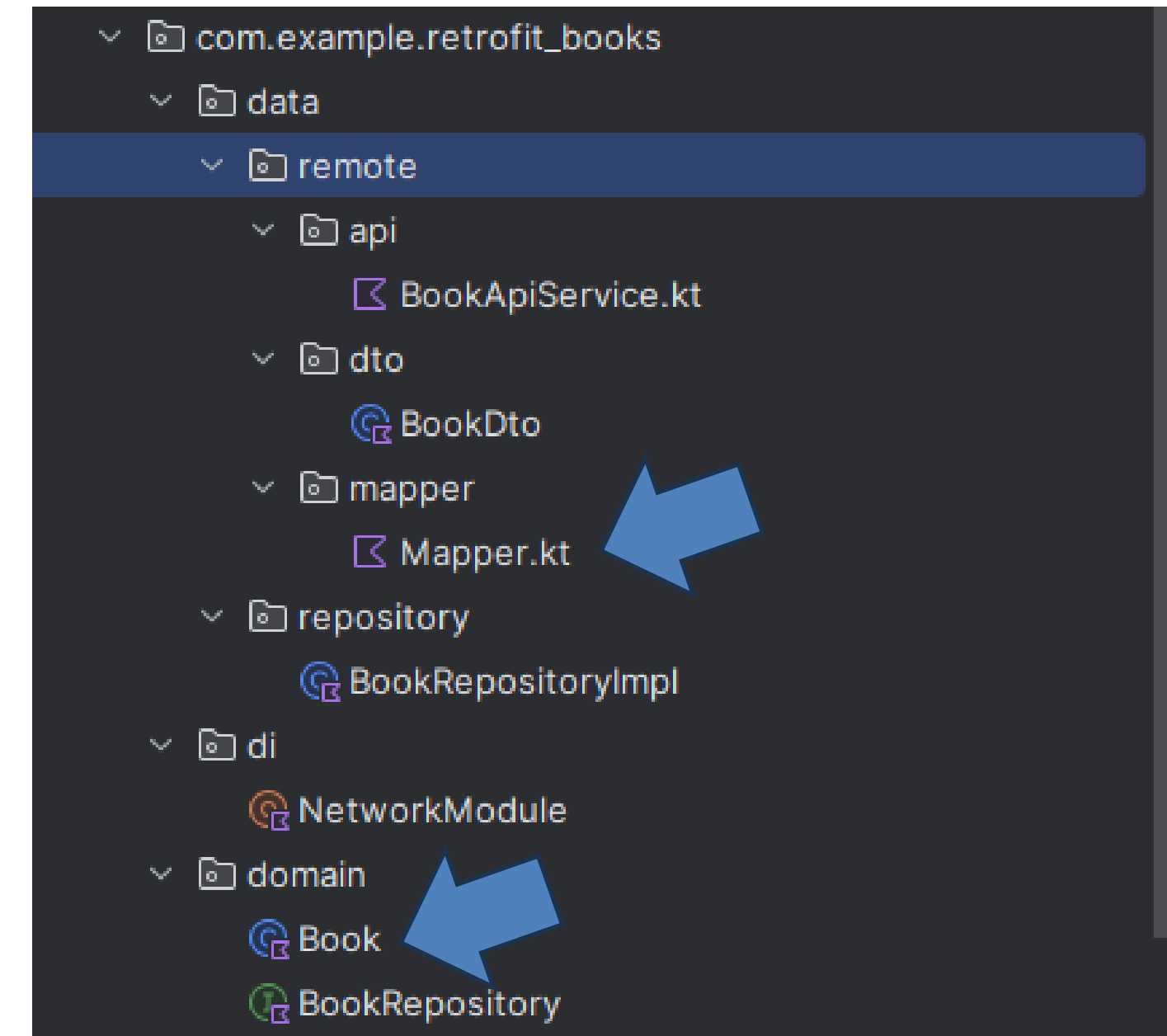
```
BookDto.kt x
1 package com.example.retrofit_books.data.remote.dto
2
3 import com.example.retrofit_books.domain.Book
4
5 data class BookDto(
6     val id: Int?,
7     val tittle: String,
8     val description: String,
9     val author: String,
10 )
11
```



Books App - Code Review (Book Domain and Mapper)

```
BookDto.kt x
1 package com.example.retrofit_books.data.remote.dto
2
3 import com.example.retrofit_books.domain.Book
4
5 data class BookDto(
6     val id: Int?,
7     val tittle: String,
8     val description: String,
9     val author: String,
10 )
11
```

```
BookDto.kt Mapper.kt x
1 package com.example.retrofit_books.data.remote.mapper
2
3 import com.example.retrofit_books.data.remote.dto.BookDto
4 import com.example.retrofit_books.domain.Book
5
6 // mapper
7 fun BookDto.toDomain() = Book(id, tittle, description, author)
8 fun Book.toDto() = BookDto(id, title, description, author)
9
```



Books App - Code Review (Book Repository)

```
BookRepository.kt x
1 package com.example.retrofit_books.domain
2
3
4 interface BookRepository {
5
6     /** List all books of the current group. */
7     suspend fun list(): List<Book>
8
9     /** Add a new book (the backend generates the id). */
10    suspend fun add(book: Book): Book
11
12    /** Replace the book whose id matches `book.id`. */
13    suspend fun update(book: Book): Book
14
15    /** Delete the book by id. */
16    suspend fun delete(id: Int)
17 }
```

```
@Singleton
class BookRepositoryImpl @Inject constructor(
    private val api: BookApiService
) : BookRepository {

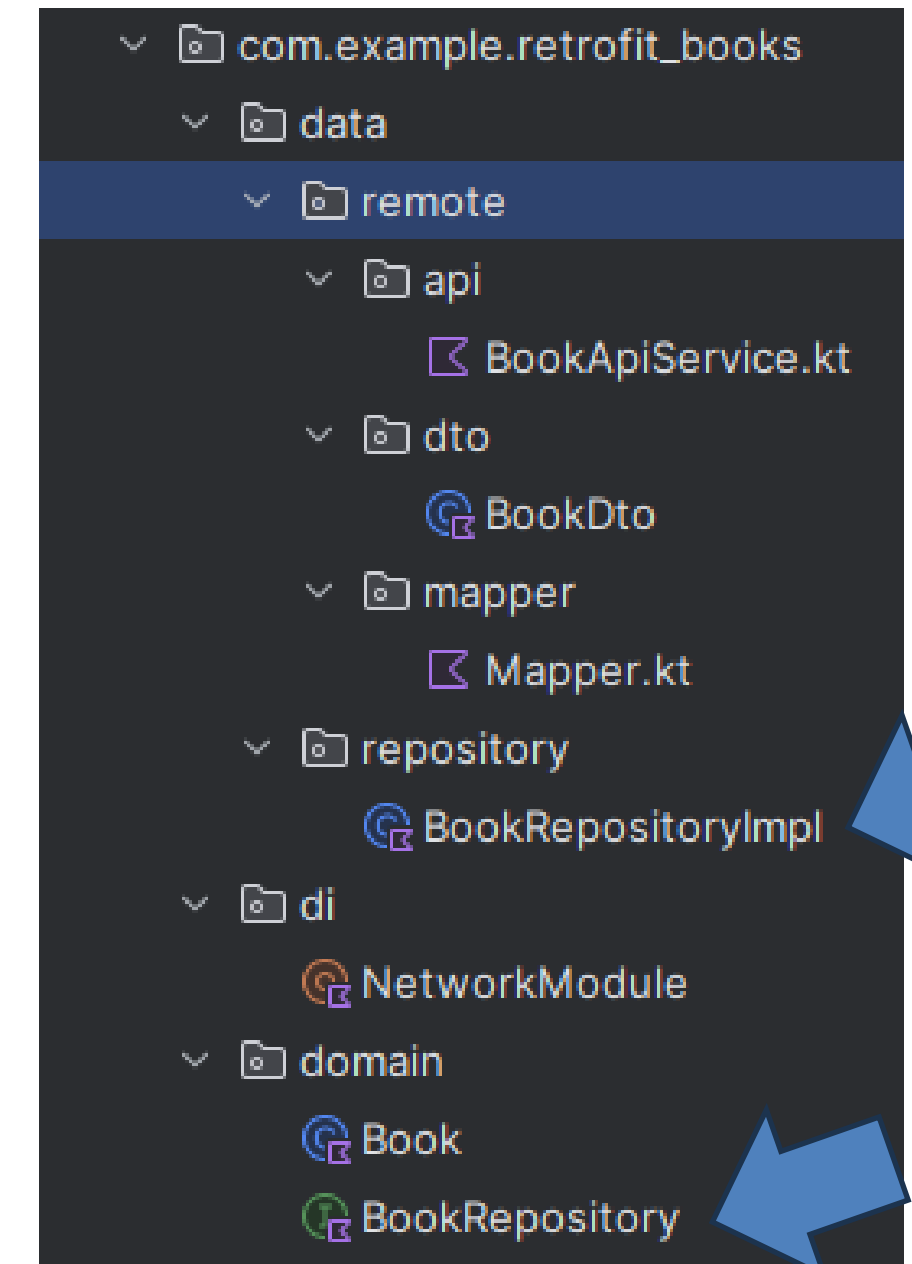
    private val gid = "G01" // hard-coded group

    override suspend fun list(): List<Book> =
        api.getBooks(gid).map { it.toDomain() }

    override suspend fun add(b: Book): Book =
        api.createBook(gid, b.toDto()).book!!.toDomain()

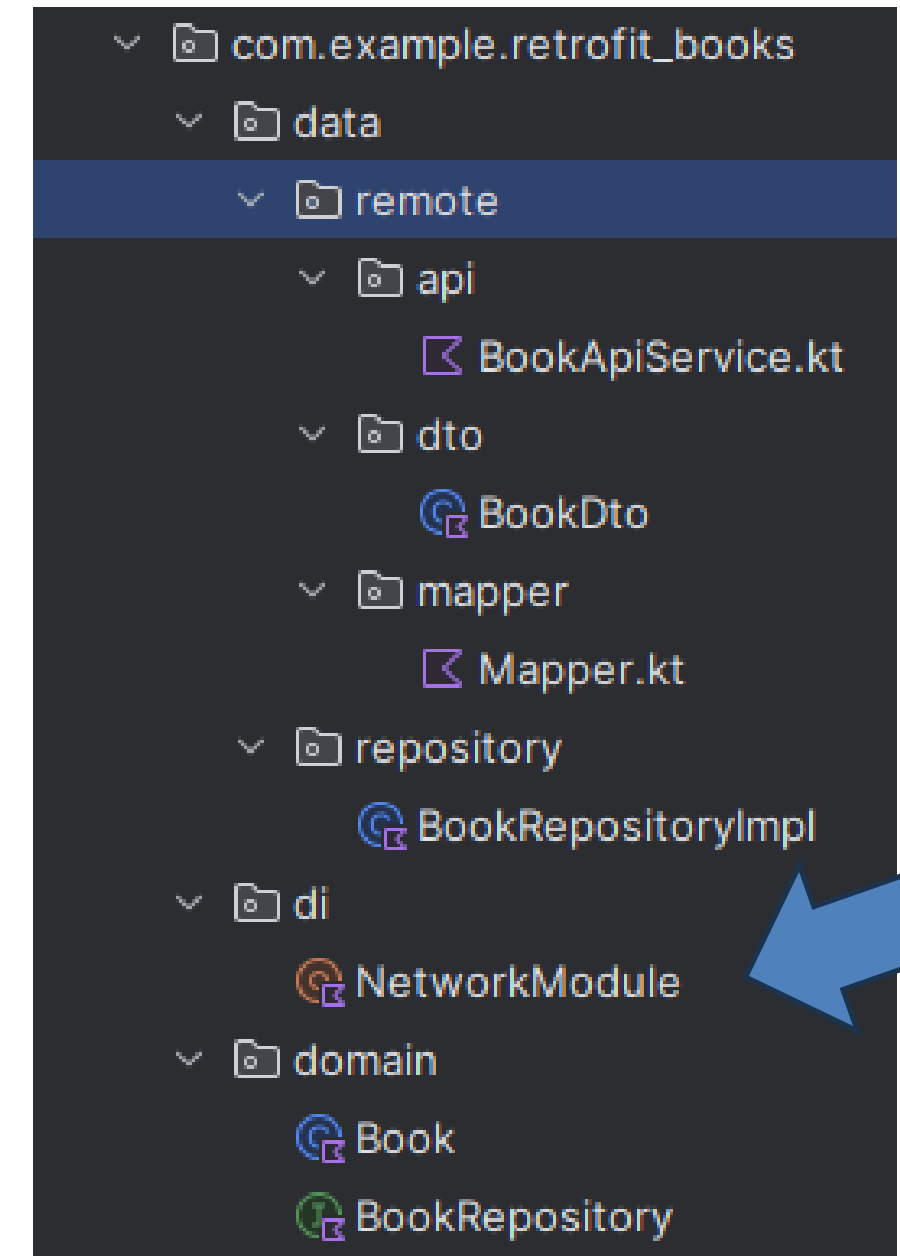
    override suspend fun update(b: Book): Book =
        api.updateBook(gid, b.id!!, b.toDto()).book!!.toDomain()

    override suspend fun delete(id: Int) {
        api.deleteBook(gid, id)
    }
}
```

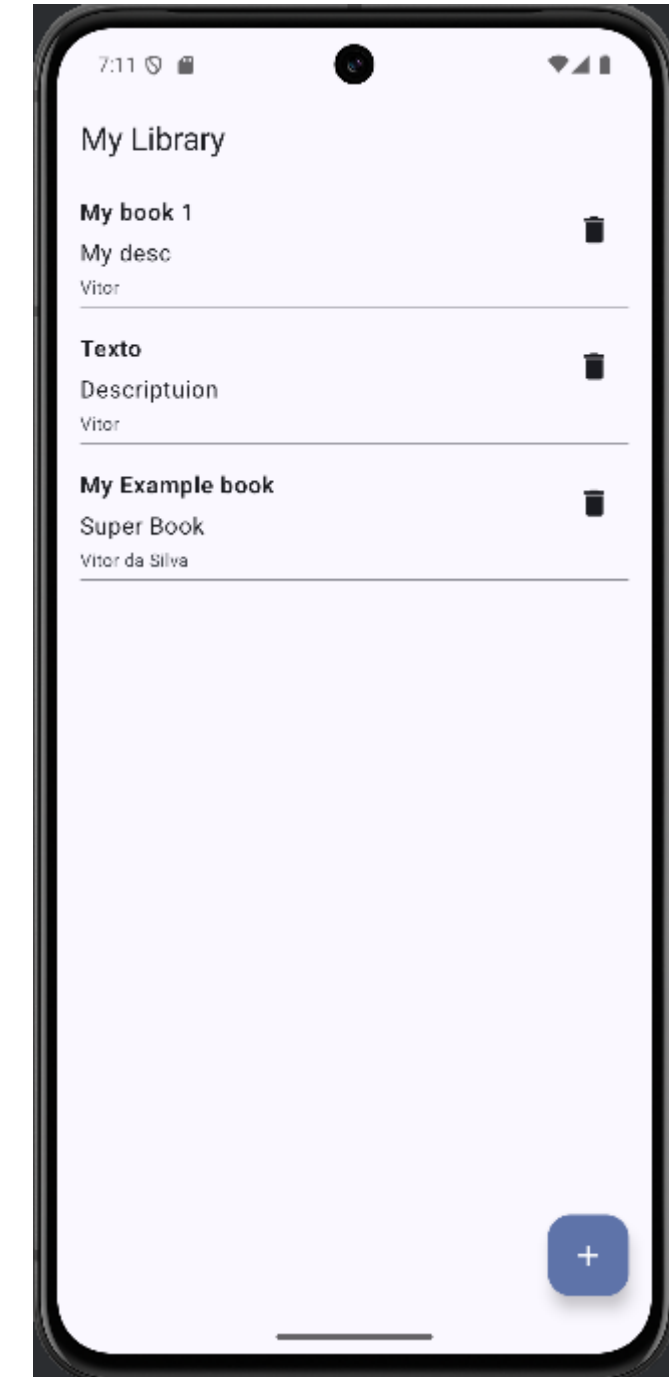
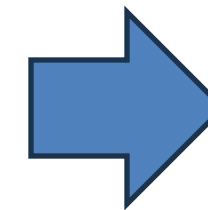
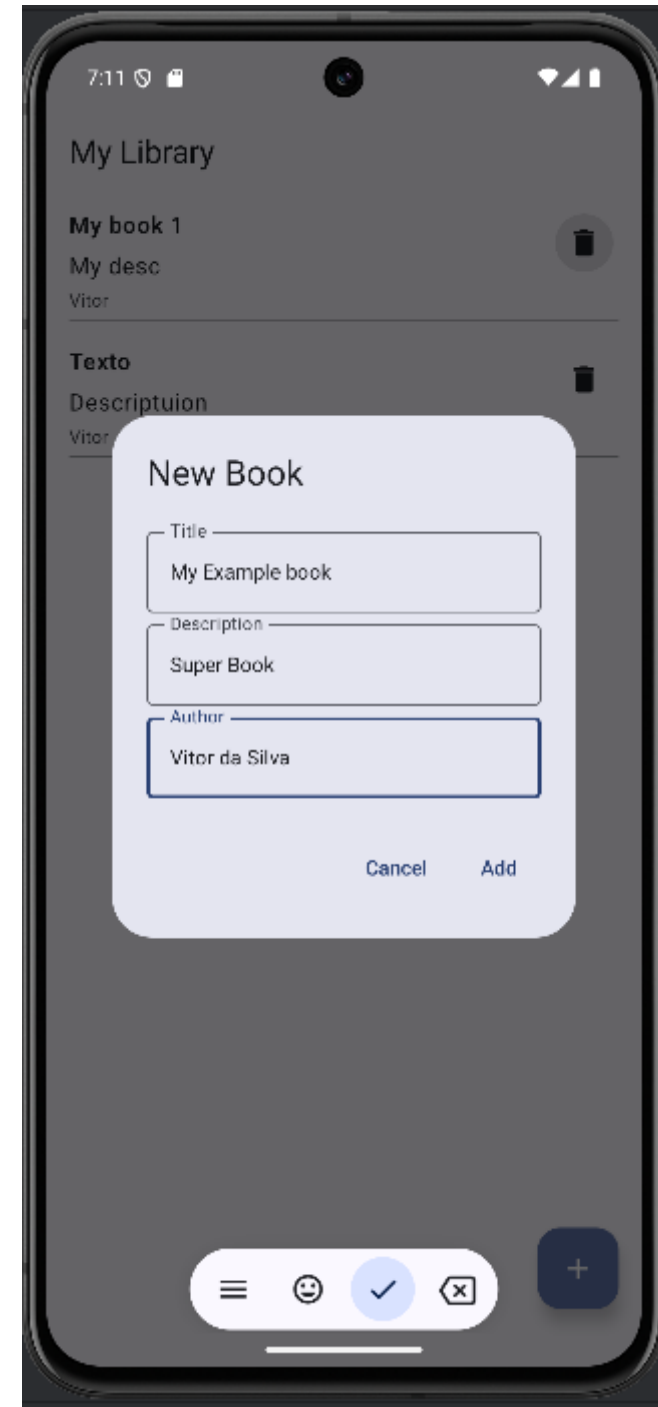
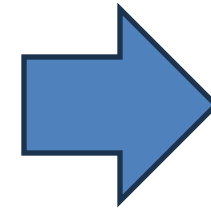
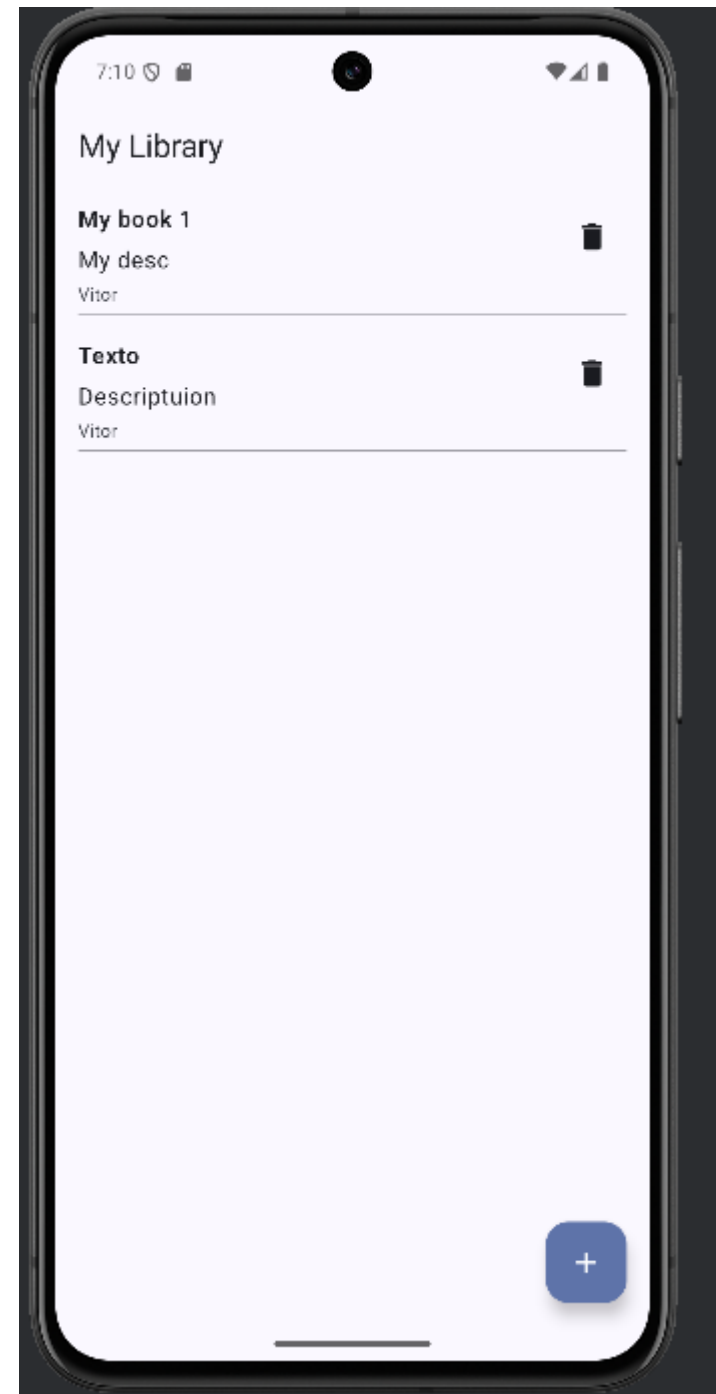


Books App - Code Review (Dependency Injection using HILT)

```
BookRepository.kt  BookRepositoryImpl.kt  NetworkModule.kt x
16  @Module
17  @InstallIn(SingletonComponent::class)
18  object NetworkModule {
19
20      @Provides
21      @Singleton
22      fun provideRetrofit(): Retrofit =
23          Retrofit.Builder()
24              .baseUrl("http://13.39.162.212:8080/")
25              .addConverterFactory(GsonConverterFactory.create())
26              .client(
27                  OkHttpClient.Builder()
28                      .addInterceptor(HttpLoggingInterceptor().apply {
29                          level = HttpLoggingInterceptor.Level.BODY
30                      })
31                      .build()
32              )
33              .build()
34
35      @Provides
36      @Singleton
37      fun provideBookRepository(api: BookApiService): BookRepository =
38          BookRepositoryImpl(api)
39
40      @Provides
41      @Singleton
42      fun provideBookApi(retrofit: Retrofit): BookApiService =
43          retrofit.create(BookApiService::class.java)
44
45  }
```



Books App - Code Review (Execution)



Homework (T04)

Homework - Books App

- Individual

Mostrar tarea...

▼ Configuración de "T04 - Implementación de la función Editar (Retrofit)"

Creado por:	Vitor Luiz Da Silva Verbel
Fecha de creación	5 may. 2026 23:10
Abierta	5 may. 2026 23:05
Entregar	17 may. 2026 23:55
Fecha límite	17 may. 2026 23:55
Modificado por el/la docente	5 may. 2026 23:14
Entrega de grupo	No
Para	Todas las secciones/grupos
Formato de envío	Editor y adjuntos
Calificación	Aprobado / Suspendido.
Alerta:	No
Cláusula de veracidad	No

Instrucciones de la tarea

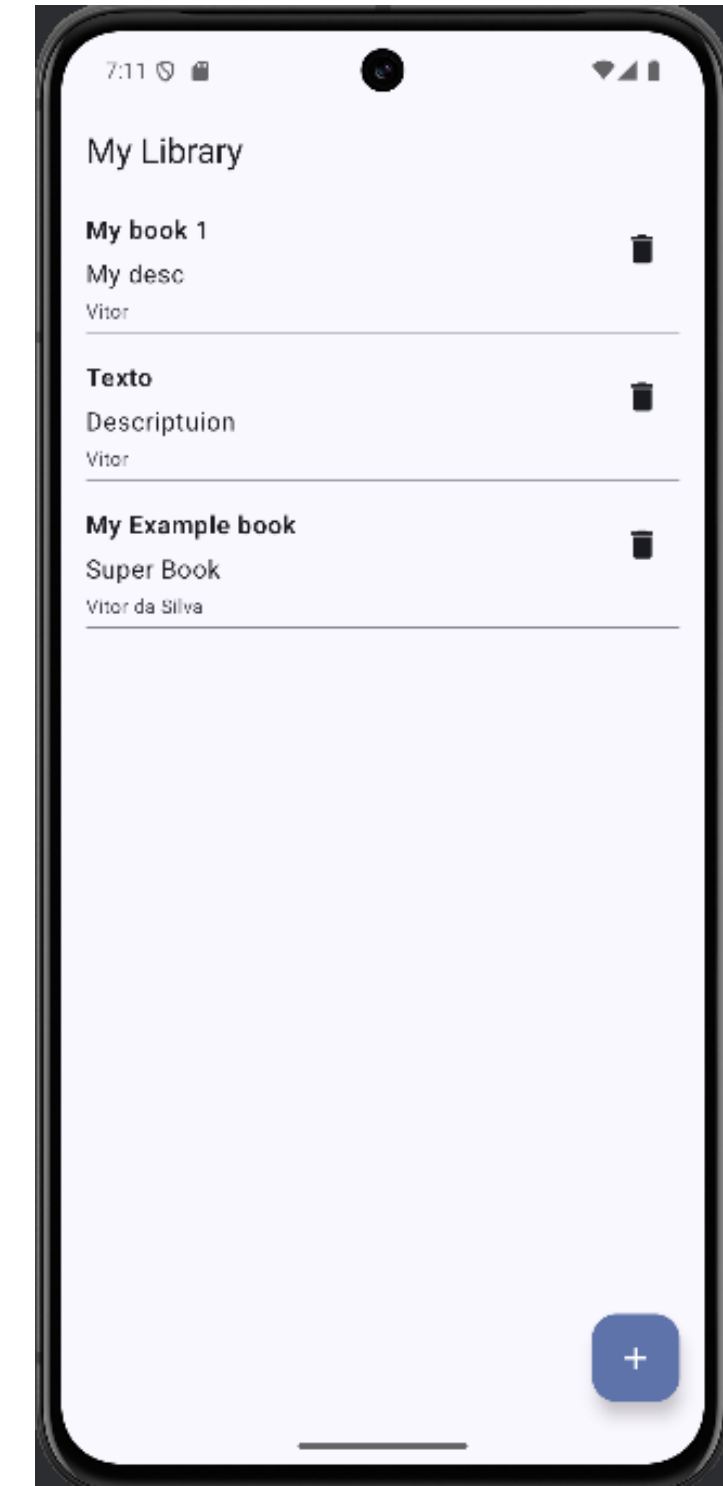
En esta actividad individual, deberás implementar la funcionalidad de **editar libros** en la aplicación de ejemplo `books-retrofit`.

- Asegúrate de que el usuario pueda modificar los datos de un libro ya existente.
- Usa como `groupId` tu **primer nombre y el primer apellido en minúsculas**. Ejemplo: vitordasilva

Recursos adicionales para la tarea

 [retrofitbooks.zip](#) (648 KB; 5 may. 2026 23:09)

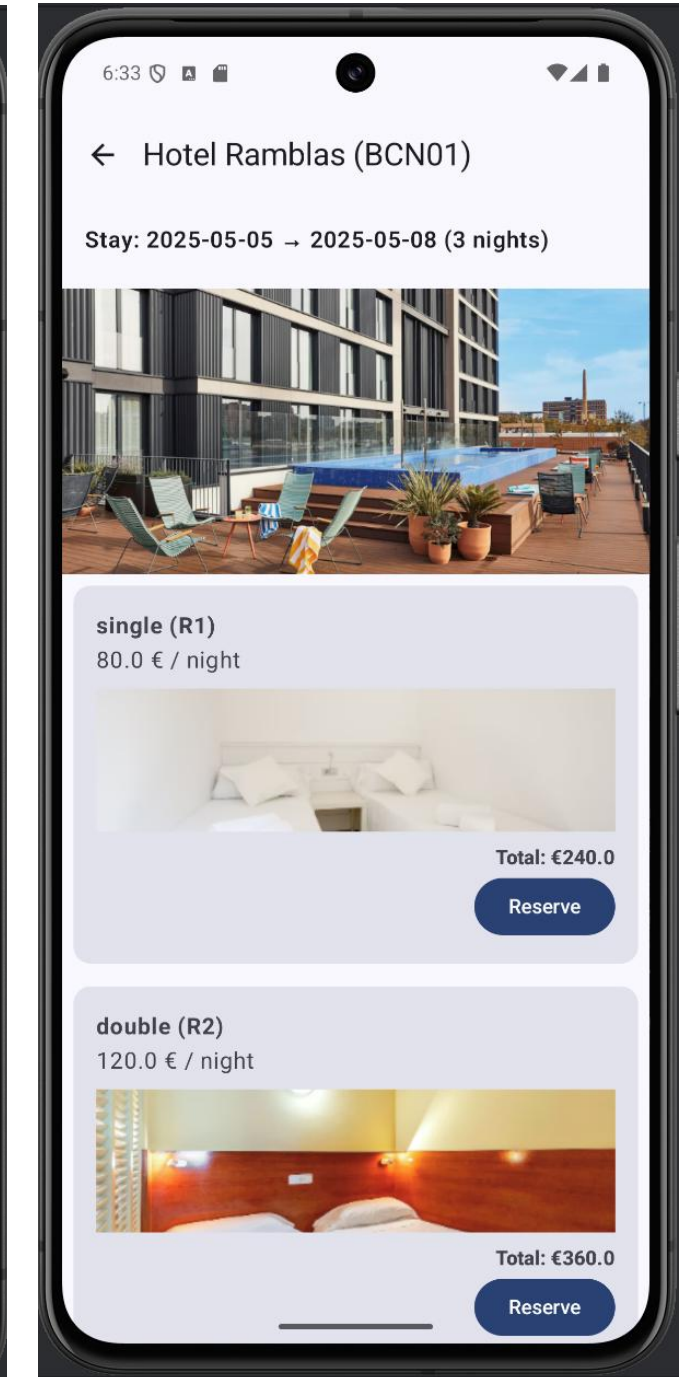
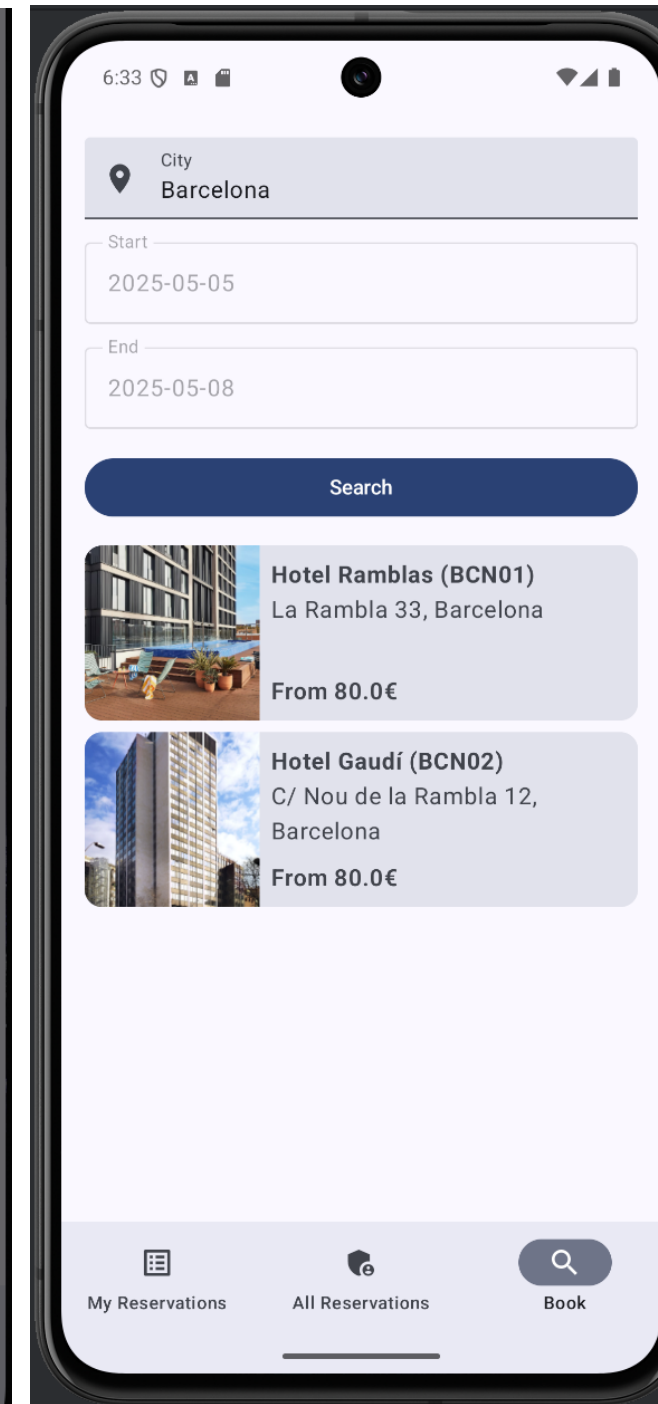
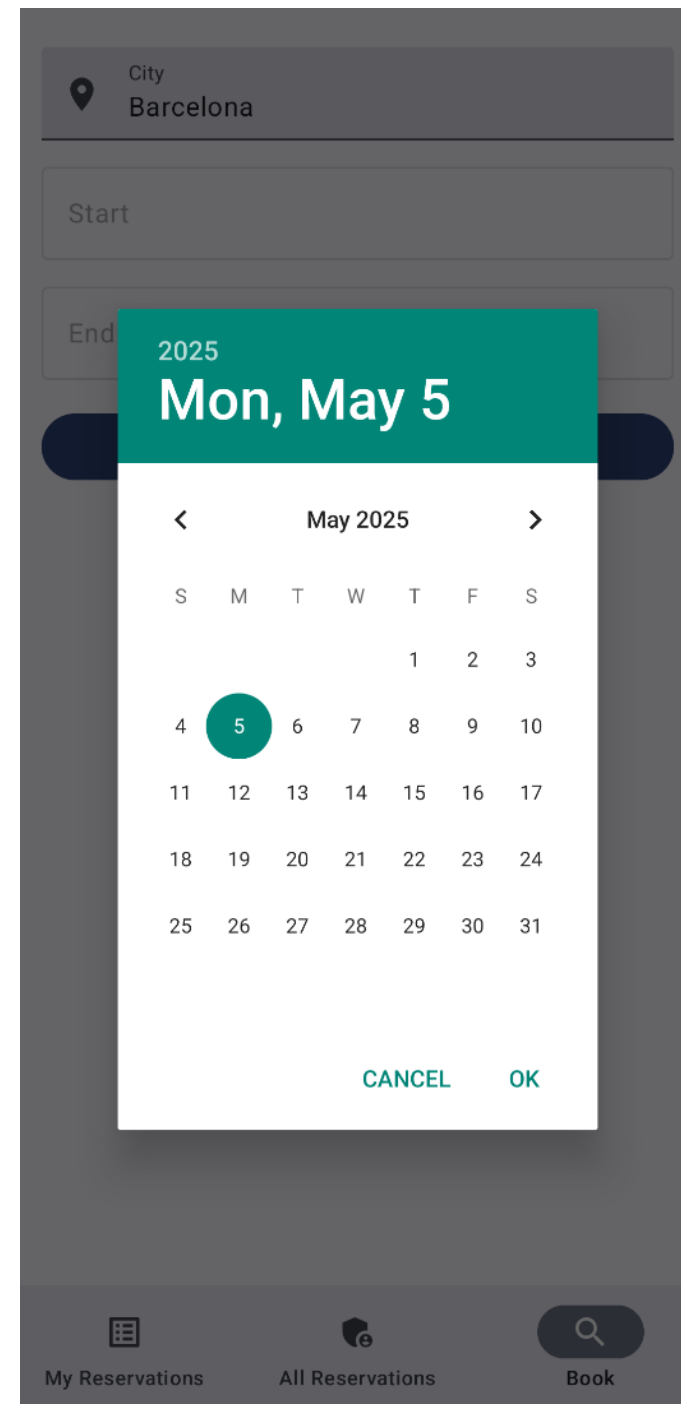
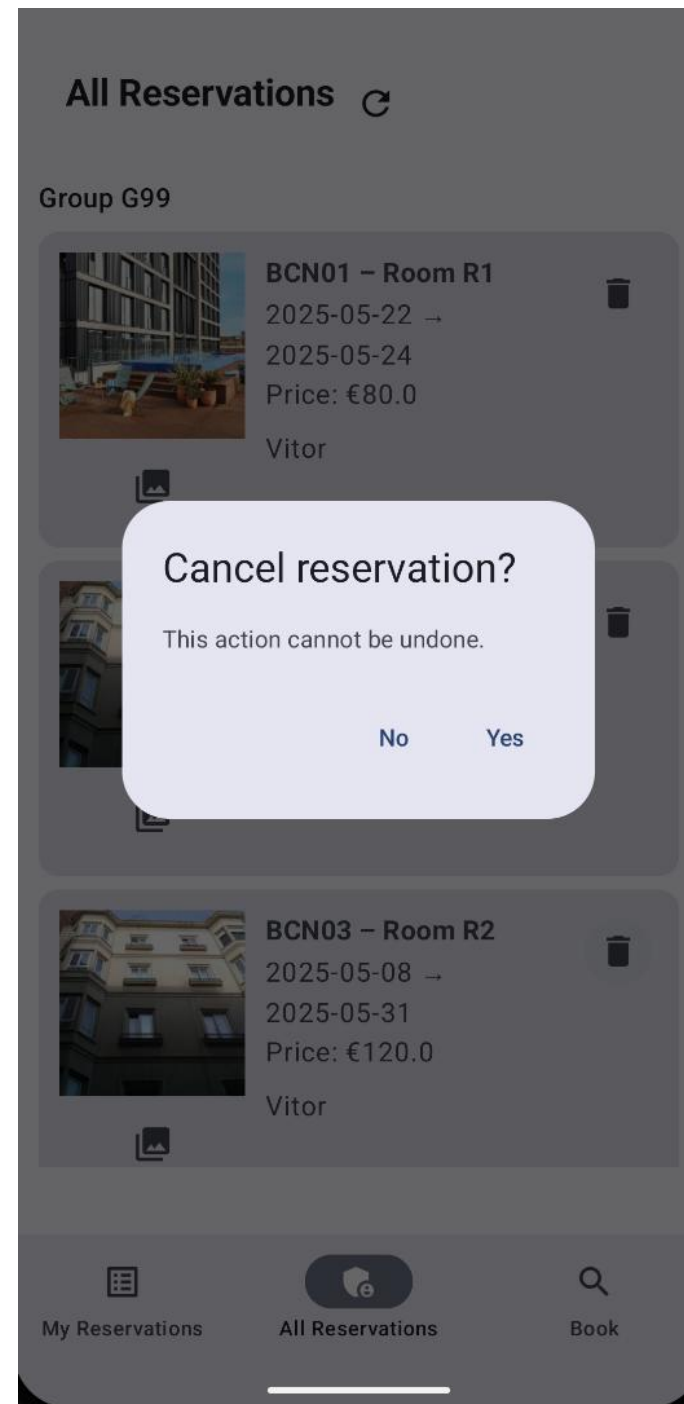
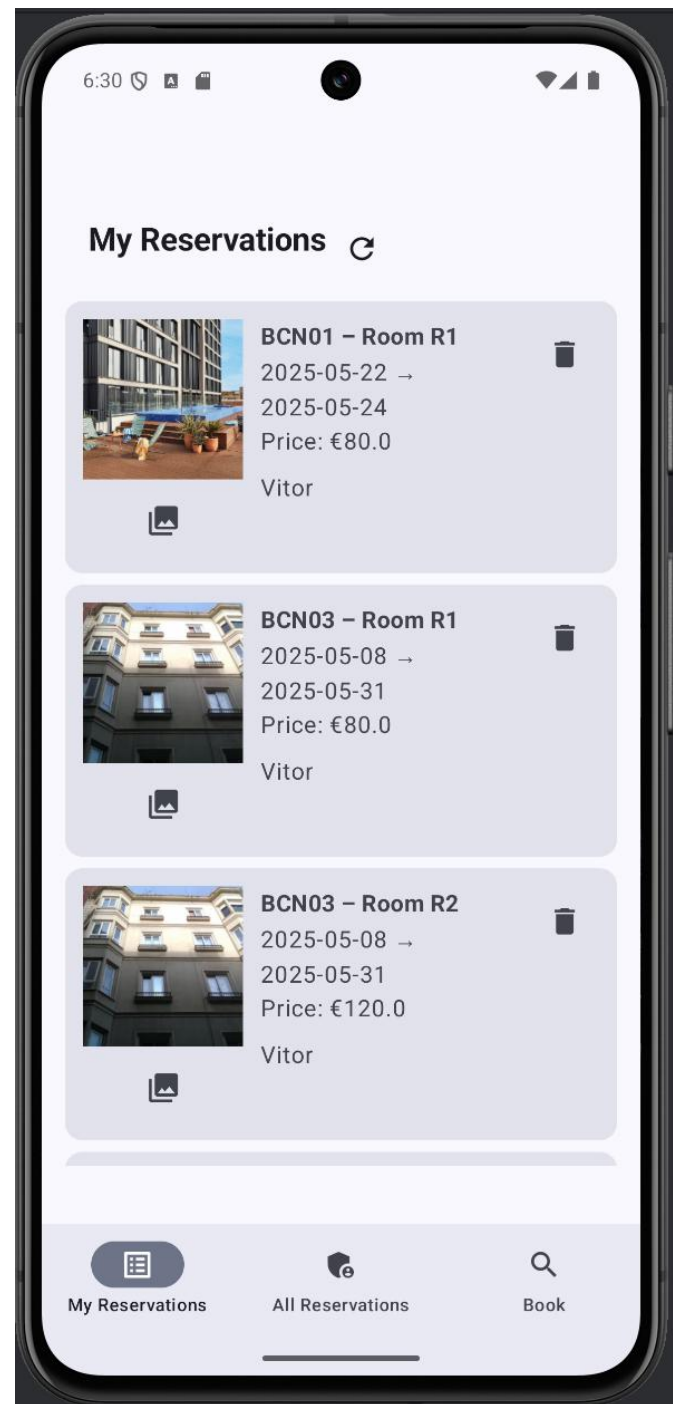
► Vista de estudiante de la tarea "T04 - Implementación de la función Editar (Retrofit)"



Sprint 04

Sprint 04 – Example Code Review

- Download at Virtual Campus: /code/[RemotePersistence.zip](#)



Sprint 04 – Retrofit

API URL: <http://ec2-3-85-11-18.compute-1.amazonaws.com/admin> or <http://15.224.84.148:8090>

API Documentation:

- <http://15.224.84.148:8090/docs>
- <http://15.224.84.148:8090/docs>
- <http://15.224.84.148:8090/redocs>
- <http://15.224.84.148:8090/admin>

ATTENTION:
EACH GROUP MUST USE YOUR OWN GROUP ID CODE
{group_id}

API Hoteles Demo

1.0.0OAS 3.1

/openapi.json

default

GET

/hotels/{group_id}/hotels

List Hotels

POST

/hotels/{group_id}/reserve

Reserve Room

GET

/hotels/{group_id}/availability

Check Availability

POST

/hotels/{group_id}/cancel

Cancel Reservation

GET

/hotels/{group_id}/reservations

List Reservations

GET

/reservations

List All Reservations

GET

/reservations/{res_id}

Get Reservation By Id

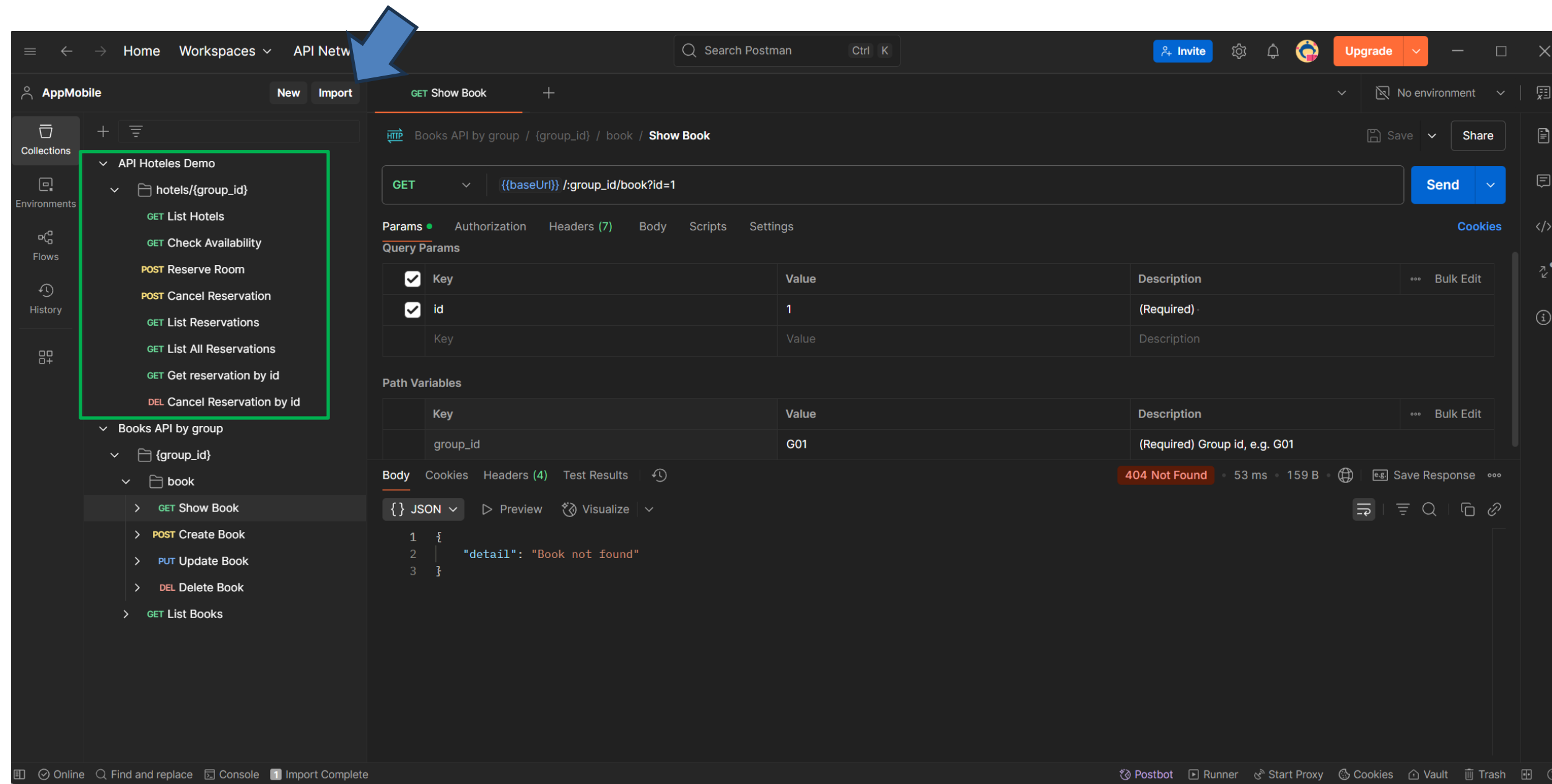
DELETE

/reservations/{res_id}

Cancel Reservation By Id

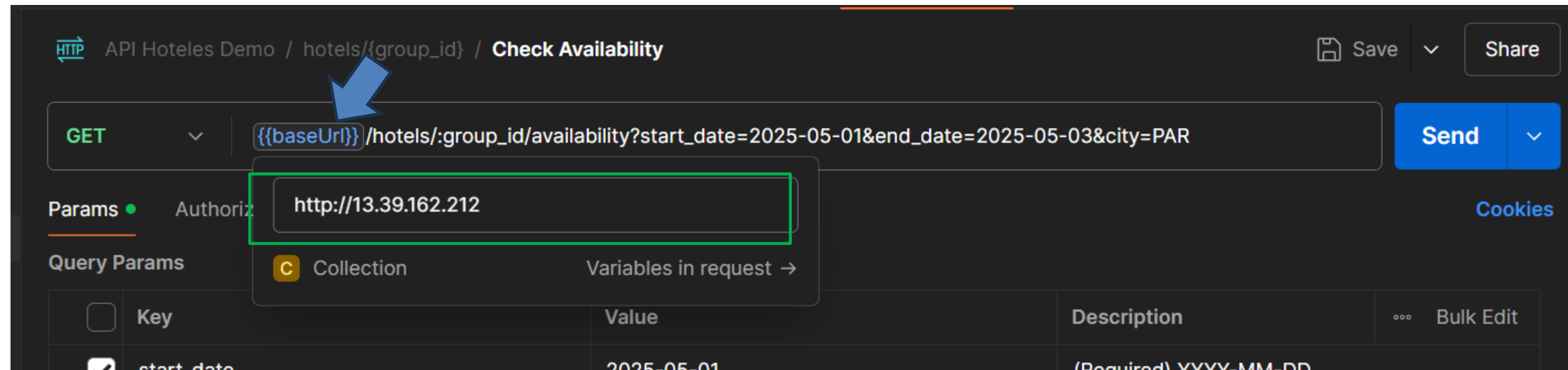
Open Postman

- Import openapi.json file: <http://15.224.84.148:8090/openapi.json>
- You should see a list of methods in Hoteles Demo

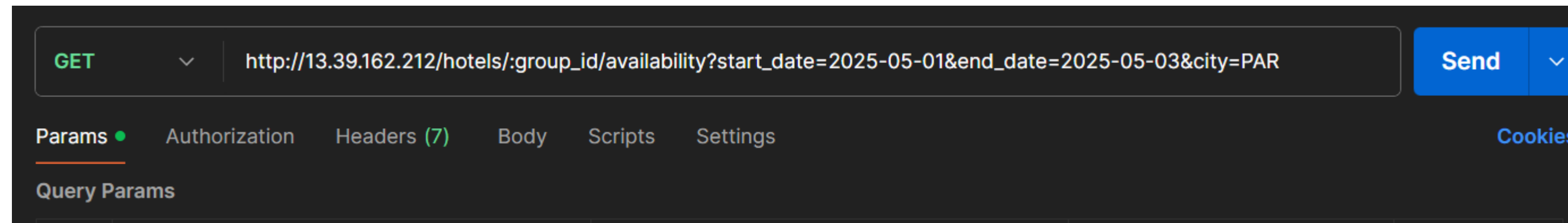


Open Postman

- Set the variable `{{baseUrl}}` to `http://15.224.84.148:8090` (Mouse over to show the option)



- Or just remove `{{baseUrl}}` and replace it directly (TRY TO NOT USE THIS METHOD)



GROUPS IDs

- From now on, use this code in your API requests:

Grupo	Estudiante 1	Estudiante 2
G01	Acosta Contreras, Sharon Elizabeth	Sánchez Macià, Joan
G02	Banqué Torné, Nel	Guarro Pérez, Gerard
G03	Beruet Morelli, Marco	Gil Cañizares, Ivan
G04	Cribillers Moncunill, Arnau	Lázaro Roca, Paul
G05	El Kabiri Ouarqui, Anouar	Mora Palomino, Eloi
G06	Escofet Presedo, Àlex	Gomà Atehortua, Samuel
G07	Garcia Huanca, Edson David	
G08	Giralt Padró, Xavier	Obando Coca, Benjamin Jonas
G09	Jaén Rico, Izan	
G10	Juárez Ontiveros, Asier	
G11	Minguez Arroyo, Adria	
G12	Alcoverro López, Guillem	Molinero Quintana, Nil
G13	Ràfols Bayot, Aleix	
G14	Senar Tejedor, Alba	
G15	Talayero Carrasco, Guillem	Àlvarez I Arisa, Oriol
G16	Rubio Palomo, Raúl	Vazquez Sanchez, Iker
G17	Farriols Segura, Guillem	

Sprint 04 – API Endpoints

- GET /hotels/{group_id}/hotels
- GET /hotels/{group_id}/availability
- POST /hotels/{group_id}/reserve
- POST /hotels/{group_id}/cancel
- GET /hotels/{group_id}/reservations

- ADMIN:
- GET /reservations
- GET /reservations/{res_id}
- DELETE /reservations/{res_id}

Sprint 04 – API Endpoints

- GET /hotels/{group_id}/hotels -> List of all hotels and available rooms

The screenshot shows a REST client interface with the following details:

- Endpoint:** GET `{{baseUrl}}/hotels/group_id/hotels`
- Path Variables:** `group_id` is set to `G99` with the description "(Required) ID del grupo, p.ej. G01".
- Status:** 200 OK, 71 ms, 3.49 KB.
- Response Body (JSON):**

```
[
  {
    "id": "BCN01",
    "name": "Hotel Ramblas",
    "address": "La Rambla 33, Barcelona",
    "rating": 4,
    "rooms": [
      {
        "id": "R1",
        "room_type": "single",
        "price": 80.0,
        "images": [
          "/images/BCN01R1.png"
        ]
      }
    ]
  },
  {
    "id": "BCN02",
    "name": "Hotel Gaudi",
    "address": "Carrer de Mallorca 401, Barcelona",
    "rating": 4.5,
    "rooms": [
      {
        "id": "R2",
        "room_type": "double",
        "price": 120.0,
        "images": [
          "/images/BCN02R2.png"
        ]
      }
    ]
  },
  {
    "id": "BCN03",
    "name": "Hotel Sagrada Família",
    "address": "Carrer de Mallorca 401, Barcelona",
    "rating": 4.8,
    "rooms": [
      {
        "id": "R3",
        "room_type": "double",
        "price": 150.0,
        "images": [
          "/images/BCN03R3.png"
        ]
      }
    ]
  }
]
```
- Code snippet:**

```
1 curl --location 'http://13.39.162.212/hotels/G99/hotels' \
2 --header 'Accept: application/json'
```

Sprint 04 – API Endpoints

- GET /hotels/{group_id}/hotels -> List of all hotels and available rooms

- City Prefixes: bcn, par and lon.

- Hotels:

- barcelona (**bcn**)

- BCN01 -> Rooms: R1, R2, R3
 - BCN02 -> Rooms: R1, R2, R3
 - BCN03 -> Rooms: R1, R2, R3 (>2 images)

- paris (**par**)

- PAR01 -> Rooms: R1, R2, R3
 - PAR02 -> Rooms: R1, R2, R3
 - PAR03 -> Rooms: R1, R2, R3 (>2 images)

- Londres (**lon**)

- LON01 -> Rooms: R1, R2, R3
 - LON02 -> Rooms: R1, R2, R3
 - LON03 -> Rooms: R1, R2, R3 (>2 images)

API Hoteles Demo

hotels/{group_id}

GET List Hotels

GET Check Availability

POST Reserve Room

POST Cancel Reservation

GET List Reservations

GET List All Reservations

GET Get reservation by id

DEL Cancel Reservation by id

Books API by group

API Hoteles Demo / hotels/{group_id} / List Hotels

Save

Share

GET

{{baseUrl}} /hotels/{group_id}/hotels

Send

Params

Authorization

Headers (7)

Body

Scripts

Settings

Pre-request

1 var template = `<canvas id="myChart" height="75"></canvas>

Post-response

Body

Cookies

(4)

Test Results

200 OK

71 ms

3.49 KB

Save Response

{}

JSON

Preview

Visualization

	id	name	address	rating	rooms				image_url		
0	BCN01	Hotel Ramblas	La Rambla 33, Barcelona	4	id	room_type	price	images	/images/BCN01.png		
					0	R1	single	80		0	/images/BCN01R1.png
					1	R2	double	120		0	/images/BCN01R2.png
					2	R3	suite	200		0	/images/BCN01R3.png
1	BCN02	Hotel Gaudí	C/ Nou de la Rambla 12, Barcelona	3	id	room_type	price	images	/images/BCN02.png		
					0	R1	single	80		0	/images/BCN02R1.png
					1	R2	double	120		0	/images/BCN02R2.png
					2	R3	suite	200		0	/images/BCN02R3.png
2	BCN03	Hotel Sagrada Família	C/ Còrsega 541, Barcelona	4	id	room_type	price	images	/images/BCN03.png		
					0	R1	single	80		0	/images/BCN03R1.png
					1	R2	double	120		0	/images/BCN03R2.png
					2	R3	suite	200		0	/images/BCN03R3.png
3	PAR01	Hotel Louvre	Rue de Rivoli 99, Paris	4	id	room_type	price	images	/images/PAR01.png		
					0	R1	single	80		0	/images/PAR01R1.png
					1	R2	double	120		0	/images/PAR01R2.png
					2	R3	suite	200		0	/images/PAR01R3.png
4	PAR02	Hotel Bastille	Boulevard Beaumarchais 25, Paris	3	id	room_type	price	images	/images/PAR02.png		
					0	R1	single	80		0	/images/PAR02R1.png
					1	R2	double	120		0	/images/PAR02R2.png
					2	R3	suite	200		0	/images/PAR02R3.png

Sprint 04 – API Endpoints

- GET /hotels/{group_id}/availability

- In this example:

- BCN
- 2025-05-01
- 2025-05-03

The screenshot displays a REST client interface for an API named 'API Hoteles Demo'. The selected endpoint is 'GET /hotels/{group_id}/availability'. The URL bar shows the full request: `{{baseUrl}} /hotels/:group_id/availability?start_date=2025-05-01&end_date=2025-05-03&city=BCN`. The 'Params' tab is active, showing query parameters: `start_date=2025-05-01`, `end_date=2025-05-03`, and `city=BCN`. Path variables include `group_id=G99`. The response status is '200 OK' with a response time of 70 ms and a size of 1.26 KB. The response body is shown in JSON format, indicating that hotel availability has been checked for group G99.

```
{
  "available_hotels": [
    {
      "id": "BCN01",
      "name": "Hotel Ramblas",
      "address": "La Rambla 33, Barcelona",
      "rating": 4,
      "image_url": "/images/BCN01.png",
      "rooms": [
        {
          "id": "R1",
          "room_type": "single",
          "price": 80.0,
          "images": [
            "/images/BCN01R1.png"
          ]
        },
        {
          "id": "R2",
          "room_type": "double"
        }
      ]
    }
  ]
}
```

Sprint 04 – API Endpoints

- GET /hotels/{group_id}/availability

- In this example:

- PAR
- 2025-05-01
- 2025-05-03

The screenshot displays a REST client interface with the following components:

- API Explorer:** A sidebar on the left showing the API structure. The selected endpoint is `GET /hotels/{group_id}/availability` under the `API Hoteles Demo` group.
- Request Configuration:** The main panel shows the request method as `GET` and the URL as `{{baseUrl}} /hotels/{group_id}/availability?start_date=2025-05-01&end_date=2025-05-03&city=PAR`. The `Send` button is highlighted.
- Query Params:** A table below the URL shows the query parameters:

Key	Value	Description
☒ start_date	2025-05-01	(Required) YYYY-MM-DD
☒ end_date	2025-05-03	(Required) YYYY-MM-DD
☐ hotel_id	BCN02	Filtrar por hotel específico
☒ city	PAR	
- Path Variables:** A table below the query params shows the path variables:

Key	Value	Description
group_id	G99	(Required)
- Response:** The `Body` tab shows the response status as `200 OK` with a response time of 67 ms and a size of 1.27 KB. The response body is displayed in JSON format:

```
1 {
2   "available_hotels": [
3     {
4       "id": "PAR01",
5       "name": "Hotel Louvre",
6       "address": "Rue de Rivoli 99, Paris",
7       "rating": 4,
8       "image_url": "/images/PAR01.png",
9       "rooms": [
10        {
11          "id": "R1",
12          "room_type": "single",
13          "price": 80.0,
14          "images": [
15            "/images/PAR01R1.png"
16          ]
17        },
18        {
19          "id": "R2",
20          "room_type": "double"
```

Sprint 04 – API Endpoints

- GET /hotels/{group_id}/availability (01-05 to 30-06)

The screenshot displays a REST client interface for the 'API Hoteles Demo'. The left sidebar lists various endpoints, including 'GET Check Availability'. The main panel shows the configuration for a GET request to the endpoint `/hotels/{group_id}/availability` with the following query parameters:

Key	Value	Description
☒ start_date	2025-05-01	(Required) YYYY-MM-DD
☒ end_date	2026-05-03	(Required) YYYY-MM-DD
☐ hotel_id	BCN02	Filtrar por hotel específico
☒ city	BCN	

The 'Path Variables' section shows the variable `group_id` with the value `G99`.

The response is a **400 Bad Request** with a status of 72 ms and 197 B. The response body is shown in raw format:

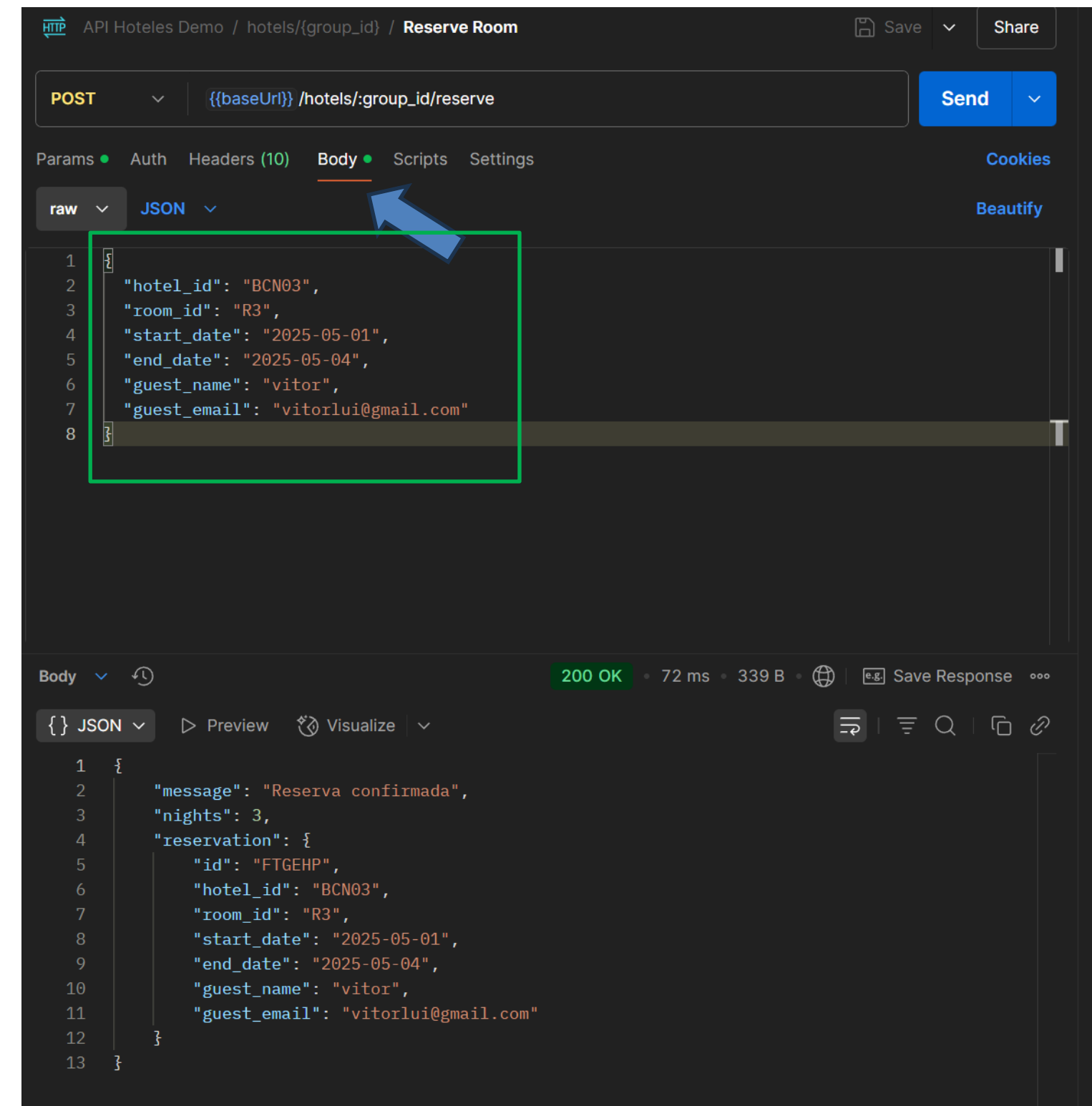
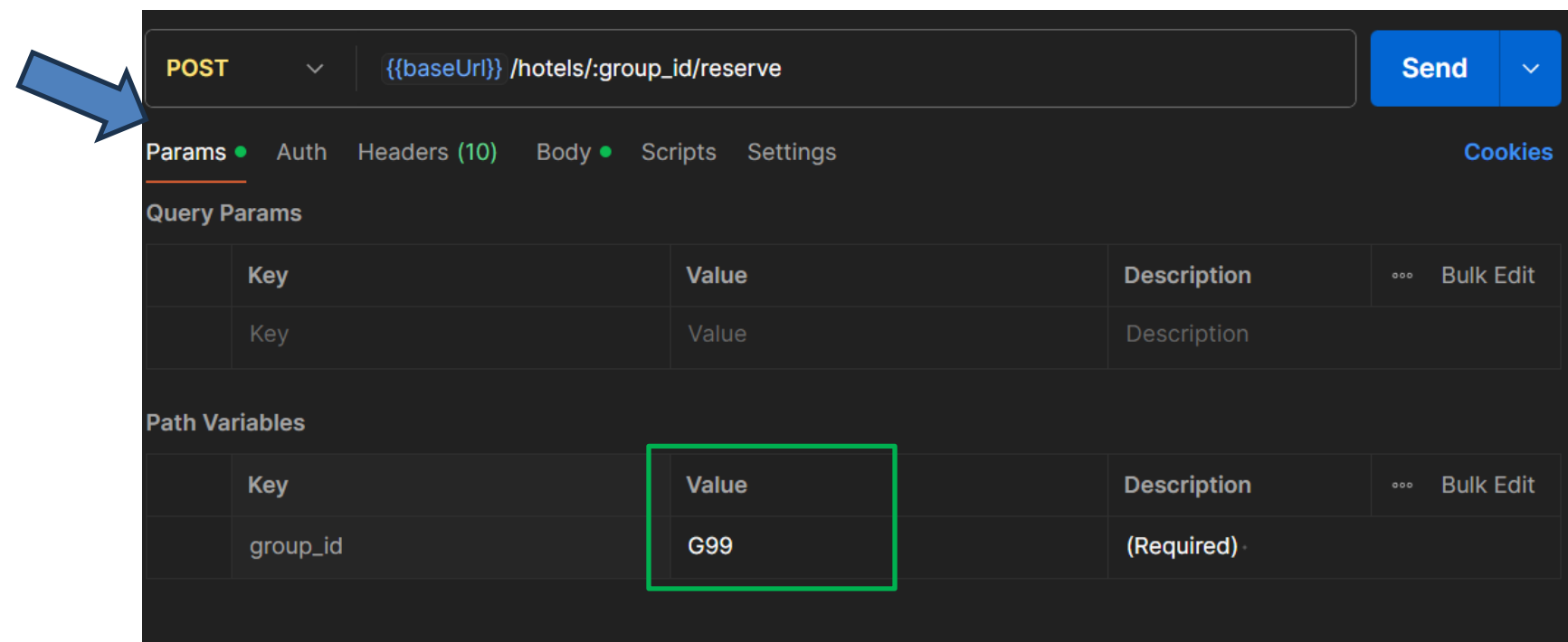
```
1 {"detail": "Sólo se permiten reservas en mayo y junio de 2025"}
```

A 'Code snippet' panel on the right shows the cURL command used for the request:

```
1 curl --location 'http://13.39.162.212/hotels/G99/availability?start_date=2025-05-01&end_date=2026-05-03&city=BCN' \
2 --header 'Accept: application/json'
```


Sprint 04 – API Endpoints

- POST /hotels/{group_id}/reserve



Sprint 04 – API Endpoints

- GET /hotels/{group_id}/list_reservations

The screenshot displays a REST client interface for an API named "API Hoteles Demo". The selected endpoint is "hotels/{group_id} / List Reservations". The HTTP method is "GET", and the URL is "{{baseUrl}} /hotels/:group_id/reservations?guest_email=vitorlui@gmail.com". A "Send" button is visible next to the URL.

Below the URL bar, there are tabs for "Params", "Auth", "Headers (7)", "Body", "Scripts", and "Settings". The "Params" tab is active, showing "Query Params".

☒	Key	Value	Description	...	Bulk Edit
☒	guest_email	vitorlui@gmail.com	Filtrar por email		
	Key	Value	Description		

Below the query params, there are "Path Variables".

	Key	Value	Description	...	Bulk Edit
	group_id	G99	(Required)		

The "Body" tab is selected, showing a "200 OK" status with a response time of 70 ms and a size of 1.63 KB. The response is in JSON format, showing a list of reservations. The JSON structure is as follows:

```
1 {
2   "reservations": [
3     {
4       "id": "USDCJM",
5       "hotel_id": "BCN01",
6       "room_id": "R1",
7       "start_date": "2025-05-22",
8       "end_date": "2025-05-24",
9       "guest_name": "Vitor",
10      "guest_email": "vitorlui@gmail.com",
11      "hotel": {
12        "id": "BCN01",
13        "name": "Hotel Ramblas",
14        "address": "La Rambla 33, Barcelona",
15        "rating": 4,
16        "image_url": "/images/BCN01.png"
17      },
18      "room": {
19        "id": "R1",
20        "room_type": "single"
21      }
22    }
23  ]
24 }
```


Sprint 04 – API Endpoints

- DELETE /reservations/{res_id}
- {res_id} is the reservation ID from List Reservations

The screenshot displays a REST client interface for an API named 'API Hoteles Demo'. The left sidebar shows a tree view of endpoints under 'hotels/{group_id}', with 'DEL Cancel Reservation by id' selected. The main panel shows the details for the 'Cancel Reservation by id' endpoint, which is a DELETE request to the URL '({baseUrl}) /reservations/reservation_id'. The 'Params' tab is active, showing a table for Path Variables with one entry: 'reservation_id' with the value 'USDCJM'. The 'Body' tab is also visible, showing a JSON response with a status of '200 OK'. The JSON response contains details about a reservation and the associated hotel and room.

API Hoteles Demo / hotels/{group_id} / Cancel Reservation by id

DELETE {{baseUrl}} /reservations/reservation_id

Params Auth Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Path Variables

Key	Value	Description	Bulk Edit
reservation_id	USDCJM	Description	

Body 200 OK 69 ms 492 B Save Response

{ JSON Preview Visualize

```
1 {
2   "id": "USDCJM",
3   "hotel_id": "BCN01",
4   "room_id": "R1",
5   "start_date": "2025-05-22",
6   "end_date": "2025-05-24",
7   "guest_name": "Vitor",
8   "guest_email": "vitorlui@gmail.com",
9   "hotel": {
10    "id": "BCN01",
11    "name": "Hotel Ramblas",
12    "address": "La Rambla 33, Barcelona",
13    "rating": 4,
14    "image_url": "/images/BCN01.png"
15  },
16  "room": {
17    "id": "R1",
18    "room_type": "single",
19    "price": 80.0,
20    "images": [
```

Sprint 04 – Retrofit (1.5 points)

API: <http://15.224.84.148:8090/docs>

Tasks

T1. Retrofit Configuration (3 Points)

- T1.1: Add Retrofit dependency and configure the HTTP client.
- T1.2: Create the necessary data models and API interfaces to access the hotel API implementing MVVM structure.
- T1.3: Create a repository layer to abstract the API usage.
- T1.4: Create a unit test to test and connection with `/hotels/{group_id}/availability`.

T2. Search and Booking Screens (4 Points)

- T2.1: Create a screen to search for hotels in **London, Paris, or Barcelona**, with city, start and end dates.
- T2.2: Show hotel and room data returned by the API (typically 3 rooms per hotel).
- T2.3: Allow users to **book a room** and save the reservation info (ID, room, hotel, price, etc) locally in the trip table or another table creating a new trip and save all reservation info.
- T2.4: Display **images** of the hotel and rooms on the booking.

Observation: follow the structure used in the example seen in class which implements:

- Search screen with a list of available hotels/rooms
- Hotel/room details screen with a button to book a room

T3. Add Images / Gallery to Trip (4 Points)

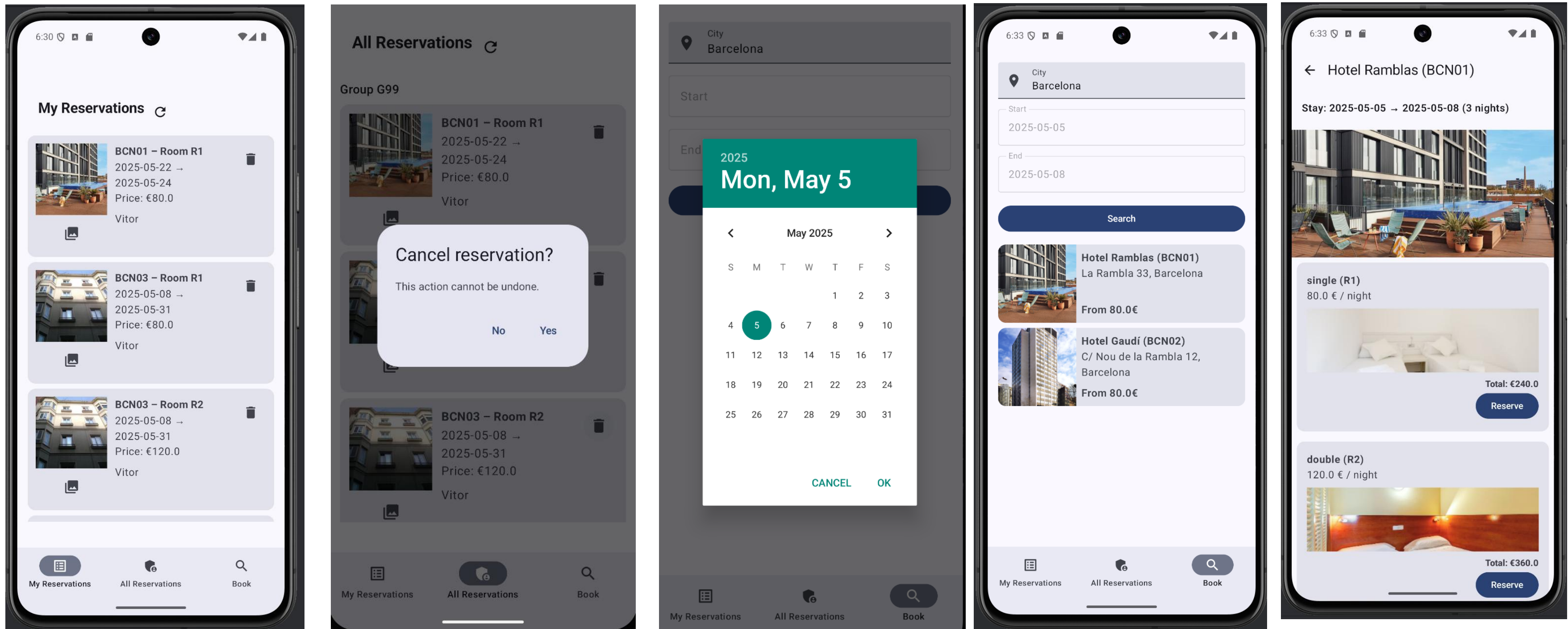
- T3.1: Allow the user to attach multiple images to a trip.
- T3.2: Save images locally in the device database or storage.
- T3.3: Display trip-specific galleries in the trip details screen.

T4. List and Cancel Reservations (3 Points)

- T4.1: Create a screen to **list all local reservations** indicating the trip related.
- T4.2: Add functionality to **delete a reservation** locally and via API (if required).
- T4.3: Show the associated hotel and room images in the list, similar to the example seen in class (RemotePersistence.zip)
- T4.4: Update the "My Trips" screen to indicate whether a trip includes a hotel reservation, and display the corresponding reservation details.

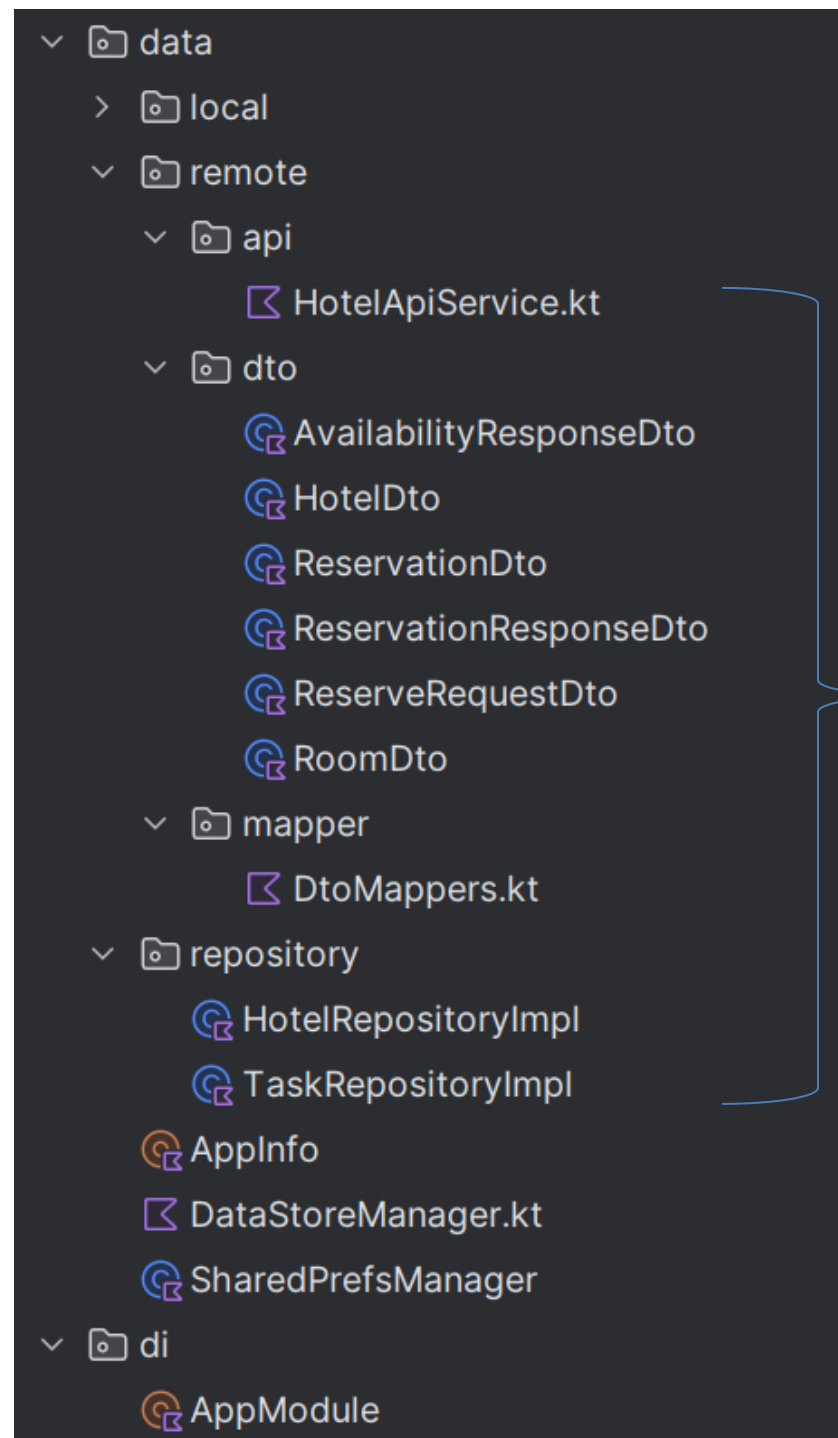
Sprint 04 – Example

- Download at Virtual Campus: /code/[RemotePersistence.zip](#)

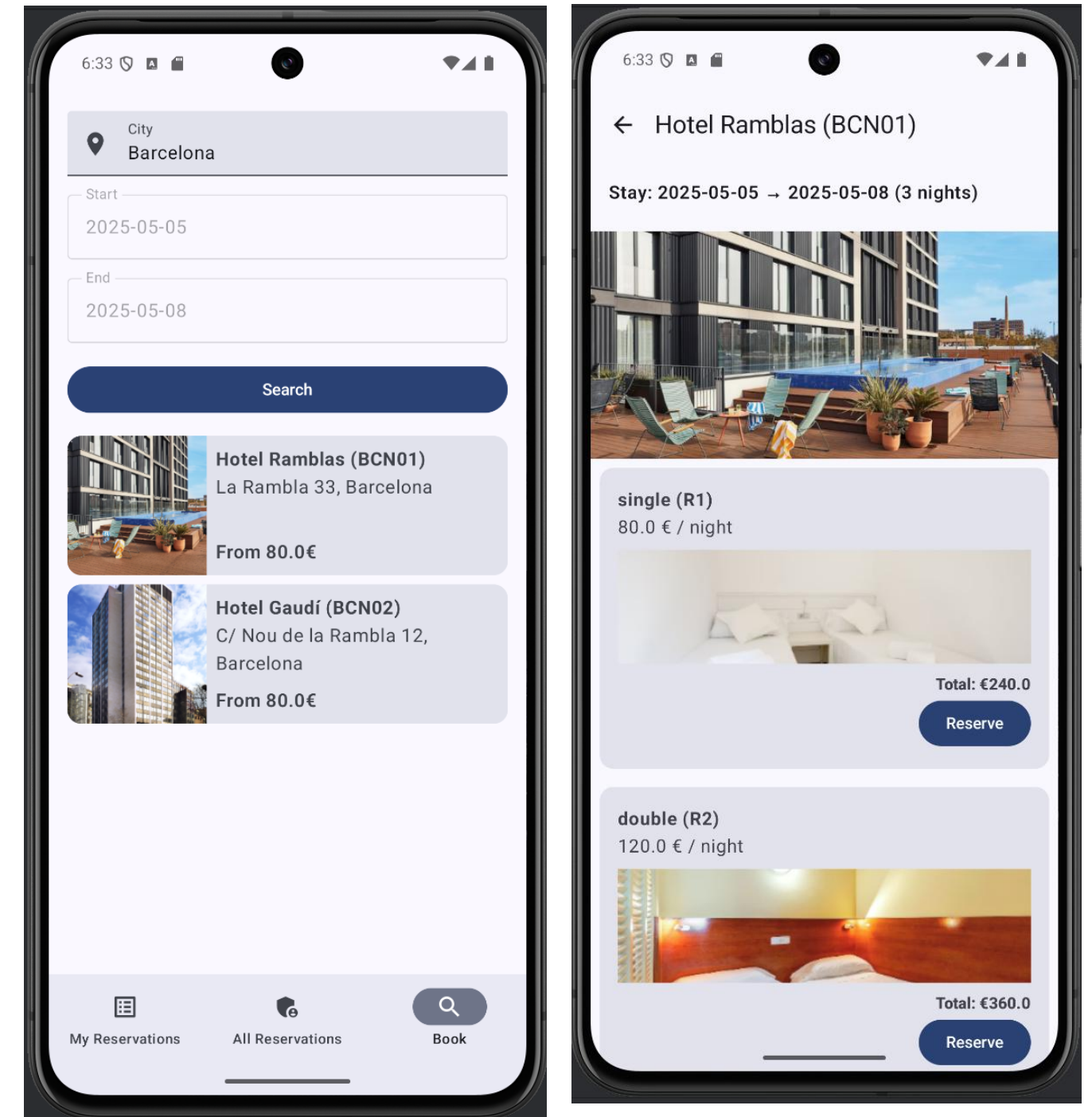


Sprint 04 – Example Code Review

- Download at Virtual Campus: /source examples/[RemotePersistence.zip](#)



NEW STRUCTURE
REMOTE PERSISTENCE



Sprint 04 – Variables Definition

- Define group_id and API_URL in build.gradle.kts

```
buildTypes {  
    debug{  
        buildConfigField( type: "String", name: "HOTELS_API_URL", value: "\"http://13.39.162.212\"")  
        buildConfigField( type: "String", name: "GROUP_ID", value: "\"G99\"")  
    }  
    release {  
        buildConfigField( type: "String", name: "GROUP_ID", value: "\"G99\"")  
        buildConfigField( type: "String", name: "HOTELS_API_URL", value: "\"http://13.39.162.212\"")  
        isMinifyEnabled = false  
        proguardFiles(  
            getDefaultProguardFile( name: "proguard-android-optimize.txt"),  
            "proguard-rules.pro"  
        )  
    }  
}
```

Sprint 04 – API definition Retrofit

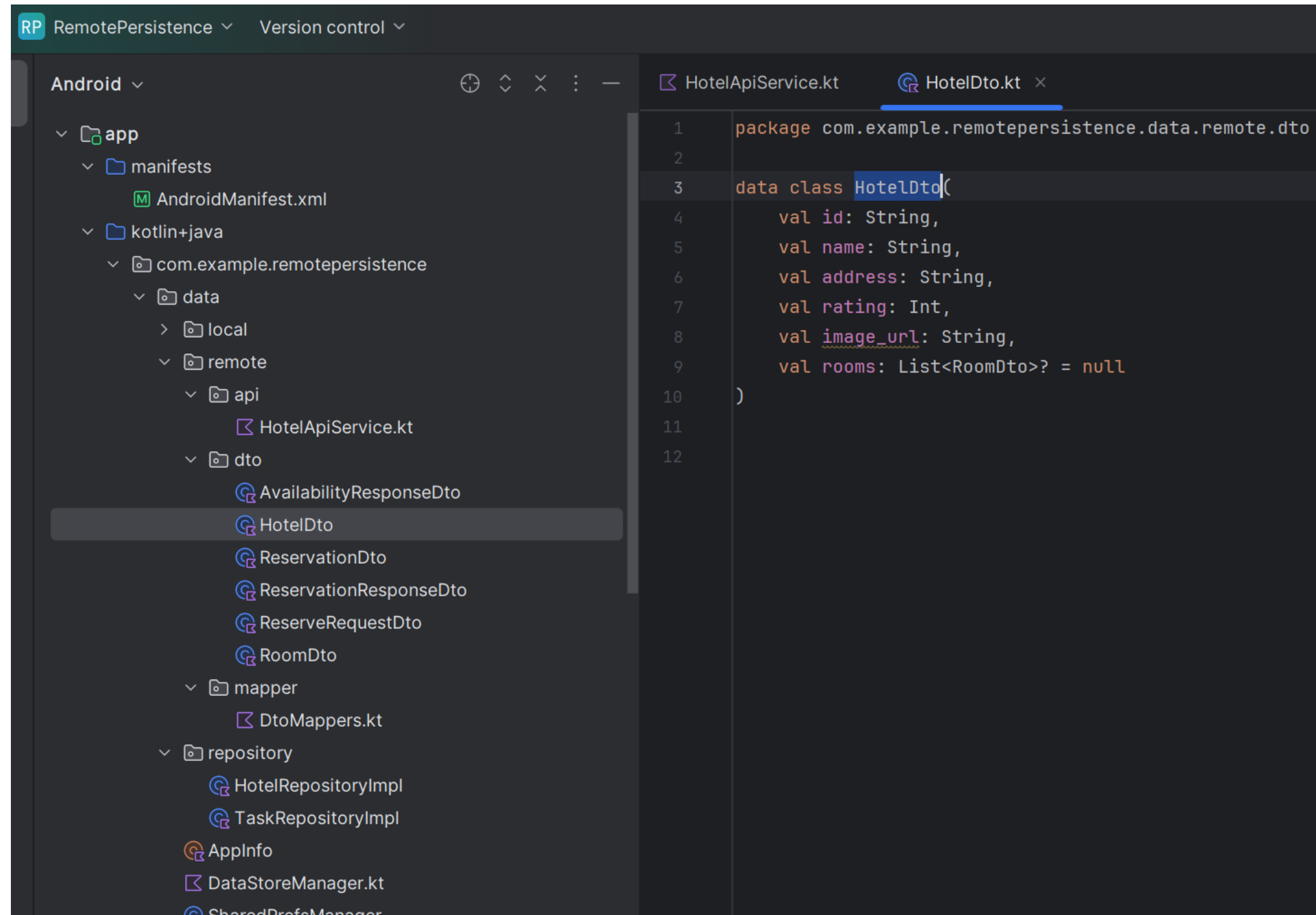
- HotelApiService.kt

```
15 interface HotelApiService {
16
17     @GET("hotels/{group_id}/hotels")
18     suspend fun getHotels(
19         @Path("group_id") groupId: String
20     ): List<HotelDto>
21
22     @GET("hotels/{group_id}/availability")
23     suspend fun getAvailability(
24         @Path("group_id") groupId: String,
25         @Query("start_date") startDate: String,
26         @Query("end_date") endDate: String,
27         @Query("hotel_id") hotelId: String? = null,
28         @Query("city") city: String? = null
29     ): AvailabilityResponseDto
30
31     /* ----- Reservations by group ----- */
32
33     @POST("hotels/{group_id}/reserve")
34     suspend fun reserveRoom(
35         @Path("group_id") groupId: String,
36         @Body request: ReserveRequestDto
37     ): ReservationResponseDto
38
39     @POST("hotels/{group_id}/cancel")
40     suspend fun cancelReservation(
41         @Path("group_id") groupId: String,
42         @Body request: CancelRequestDto
43     ): ApiMessageDto
44 }
```

// same fields as Reserve
// e.g. { "message": "Reserva cancelada" }

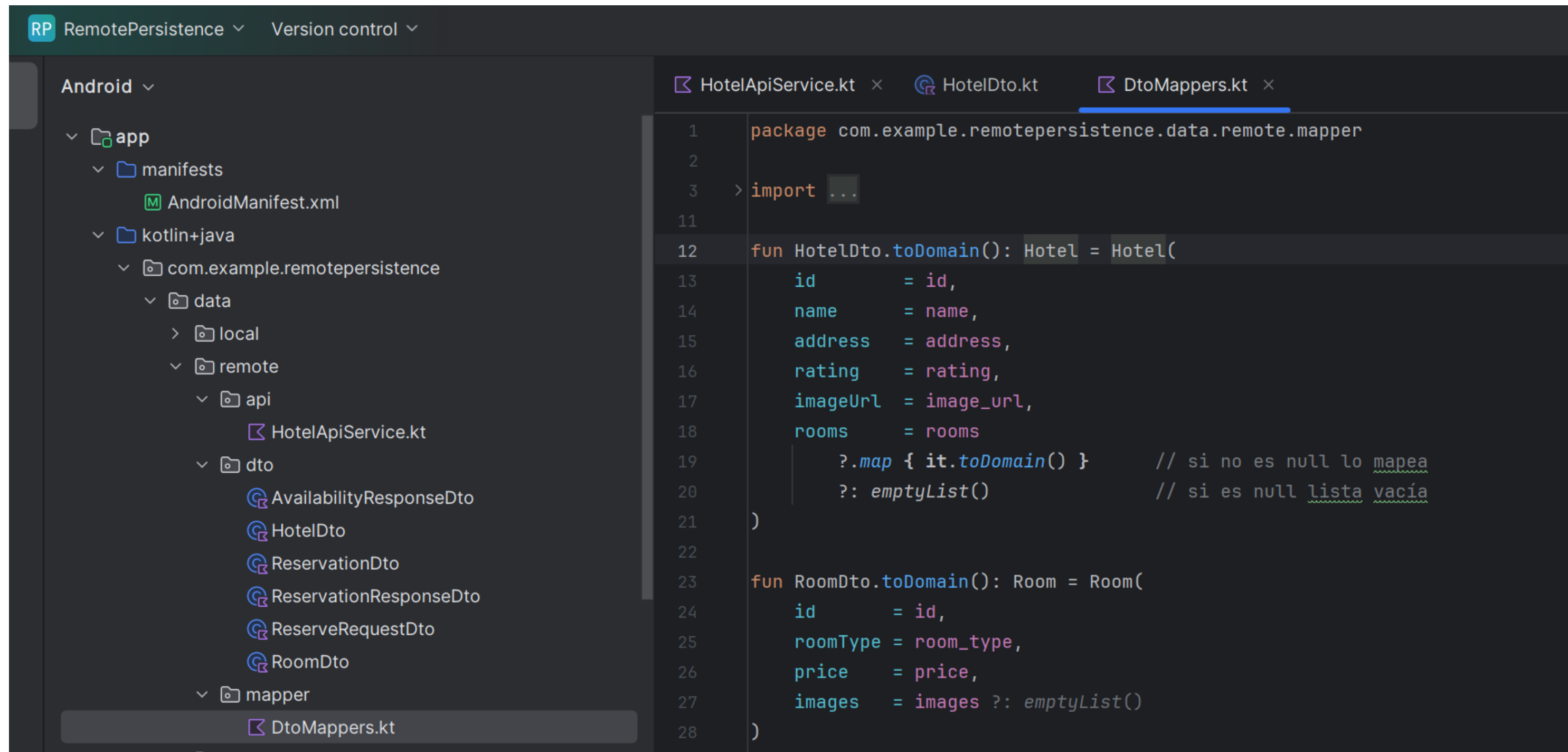
Sprint 04 – Data Transfer Objects (DTOs)

- DTOs at remote/dto



Sprint 04 – Mappers

- Mappers (DTO ↔ DOMAIN)

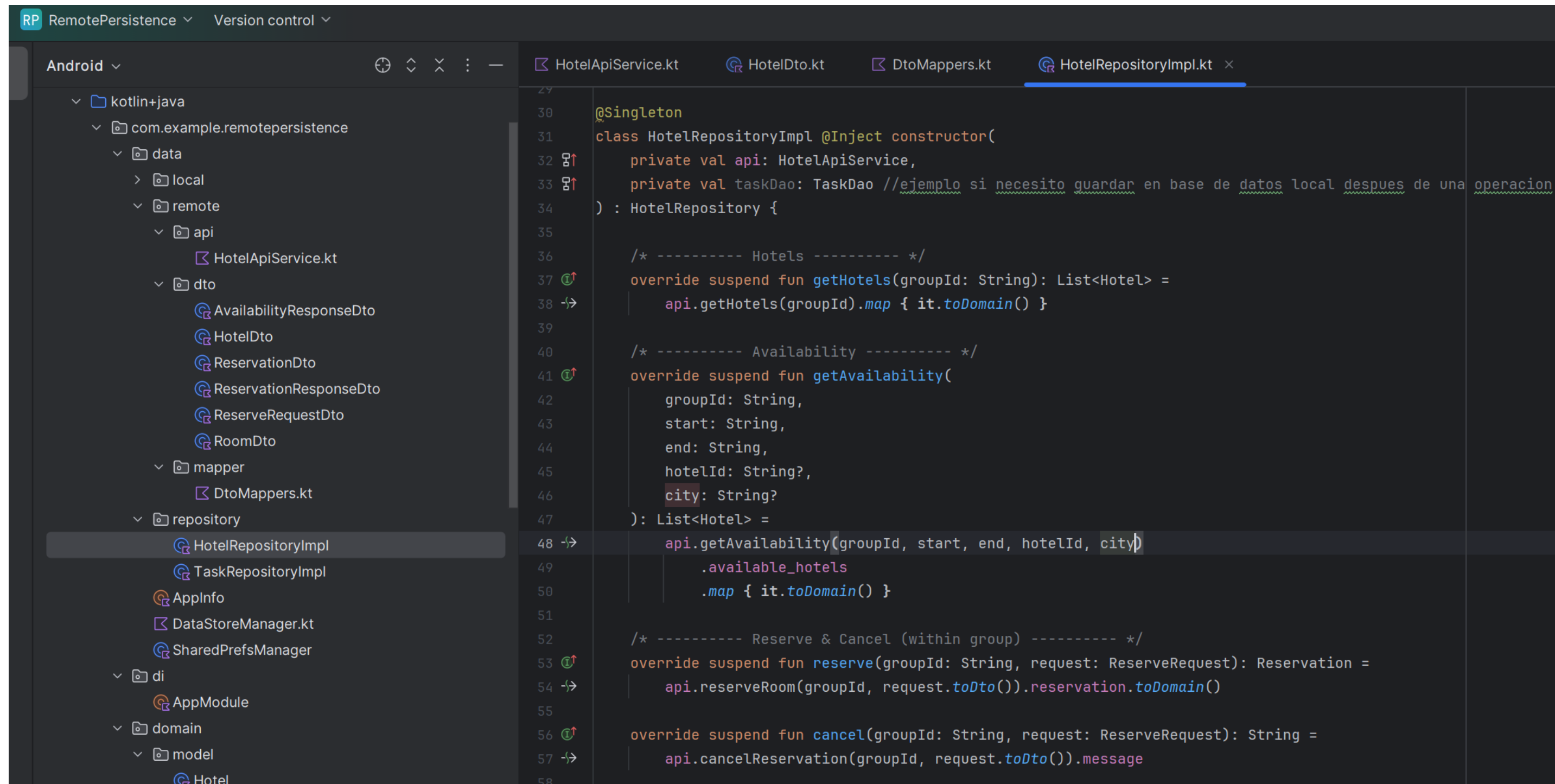


The screenshot shows an IDE with a project structure on the left and Kotlin code on the right. The project structure is for an Android app named 'RemotePersistence'. It includes a 'dto' package with several DTO classes and a 'mapper' package containing 'DtoMappers.kt'. The code in 'DtoMappers.kt' defines two functions: 'HotelDto.toDomain()' and 'RoomDto.toDomain()'. The first function maps a 'HotelDto' to a 'Hotel' domain object, and the second function maps a 'RoomDto' to a 'Room' domain object. Both functions use the 'map' function to map lists of DTOs to lists of domain objects, with a fallback to 'emptyList()' if the input is null.

```
1 package com.example.remotepersistence.data.remote.mapper
2
3 > import ...
4
5
6
7
8
9
10
11
12 fun HotelDto.toDomain(): Hotel = Hotel(
13     id = id,
14     name = name,
15     address = address,
16     rating = rating,
17     imageUrl = image_url,
18     rooms = rooms
19     ?.map { it.toDomain() } // si no es null lo mapea
20     ?: emptyList() // si es null lista vacía
21 )
22
23 fun RoomDto.toDomain(): Room = Room(
24     id = id,
25     roomType = room_type,
26     price = price,
27     images = images ?: emptyList()
28 )
```

Sprint 04 – Mappers

- Repository Implementation



```
29
30 @Singleton
31 class HotelRepositoryImpl @Inject constructor(
32     private val api: HotelApiService,
33     private val taskDao: TaskDao //ejemplo si necesito guardar en base de datos local despues de una operacion
34 ) : HotelRepository {
35
36     /* ----- Hotels ----- */
37     override suspend fun getHotels(groupId: String): List<Hotel> =
38         api.getHotels(groupId).map { it.toDomain() }
39
40     /* ----- Availability ----- */
41     override suspend fun getAvailability(
42         groupId: String,
43         start: String,
44         end: String,
45         hotelId: String?,
46         city: String?
47     ): List<Hotel> =
48         api.getAvailability(groupId, start, end, hotelId, city)
49             .available_hotels
50             .map { it.toDomain() }
51
52     /* ----- Reserve & Cancel (within group) ----- */
53     override suspend fun reserve(groupId: String, request: ReserveRequest): Reservation =
54         api.reserveRoom(groupId, request.toDto()).reservation.toDomain()
55
56     override suspend fun cancel(groupId: String, request: ReserveRequest): String =
57         api.cancelReservation(groupId, request.toDto()).message
58 }
```

Enquesta



<https://esecretaria.udl.cat/ServiciosApp>

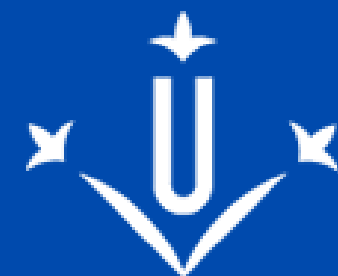
Applications for Mobile Devices

That's all

Full Name

2022 March 27

Course: 105025-2425



Universitat
de Lleida

